

© Copyright by Olusegun Sobulo, 2004

QUERY TRANSLATION FOR AN INFORMATION INTEGRATION
SYSTEM
AND BUILDING A MONITORING SYSTEM

BY

OLUSEGUN SOBULO

B.S., Illinois Institute of Technology, 2001

THESIS

Submitted in partial fulfillment of the requirements
for the degree of Master of Science in Computer Science
in the Graduate College of the
University of Illinois at Urbana-Champaign, 2004

Urbana, Illinois

Abstract

Information integration systems provide access to numerous sources through a single interface. We present a real example of building such a system over book sources on the web using a mass collaboration approach known as MOBS that leverages input from multiple users. Furthermore we discuss the different modules that comprise the system and focus in particular on the query translation module for the system. We also discuss the challenges involved in building monitoring systems and present real examples of building such systems over websites of researchers and courses. We then highlight the similarities between building information integration systems and monitoring systems and discuss monitoring systems as an application of information integration systems. In all of our discussions on the real world systems we built, we highlight the obstacles faced in building such systems and the lessons we learnt in tackling these obstacles.

To my family

Acknowledgments

First and foremost, I would like to thank my advisor, Anhai Doan. This thesis would not have been possible without his invaluable guidance. I would also like to thank the members of my research group, in particular the people I worked closely with on various projects, Robert, Rishi and Vanitha.

Last but not least I would like to thank my best friend, Omotola, for her constant support and encouragement.

Table of Contents

Chapter 1	Introduction	1
1.1	Information Integration (II) Systems	1
1.2	Common Problems	2
1.3	Thesis Overview	5
Chapter 2	Mass Collaboration to Build Systems (MOBS)	7
2.1	What is MOBS?	7
2.2	Framework	7
2.3	Incentive Models	9
Chapter 3	Applying MOBS to an II System over a Book Domain	12
3.1	System Overview	12
3.2	Source Discovery and Recognizing Query Interfaces	13
3.3	Matching Attributes of Interfaces	14
3.4	Translating User Queries	14
3.4.1	Rules Generation	15
3.4.2	Using Rules to Translate Queries	17
3.4.3	Combining and Returning Results of Queries	19
3.5	Experimental Results	20
3.6	Related Work	21
3.7	Summary	21
Chapter 4	Building a Monitoring System	22
4.1	Introduction	22
4.2	The Two Domains	23
4.3	Overview of the Monitoring System	23
4.4	Description of Modules	24
4.4.1	GUI	24
4.4.2	Configuration	24
4.4.3	Taking Snapshots	26
4.4.4	Crawler	27
4.4.5	Data Extraction	28
4.4.6	Caching	29
4.4.7	Finding Changes	30
4.4.8	Notification	31

4.4.9	Scheduler	32
4.5	Monitoring as a Variant of II Systems	32
4.6	Related Work	35
4.7	Summary	37
Appendix A Perl Code for Monitoring System		38
References		63

Chapter 1

Introduction

1.1 Information Integration (II) Systems

There are an ever-increasing number of data sources available on the web, the contents of which are only available via query interfaces. Examples of such sources include buying books on amazon.com, finding an apartment to rent on citycrybs.com, or finding what movies are playing in theaters on movies.com. For any single domain of interests, e.g. books, there are numerous sources dedicated to it. This rapid growth of distributed data sources on the web has generated much interest in building information integration systems. The goal of information integration systems is to provide a global interface to numerous data sources within a particular domain, thereby allowing users to query just the global interface rather than having to query each source individually and then manually combine the results[13].

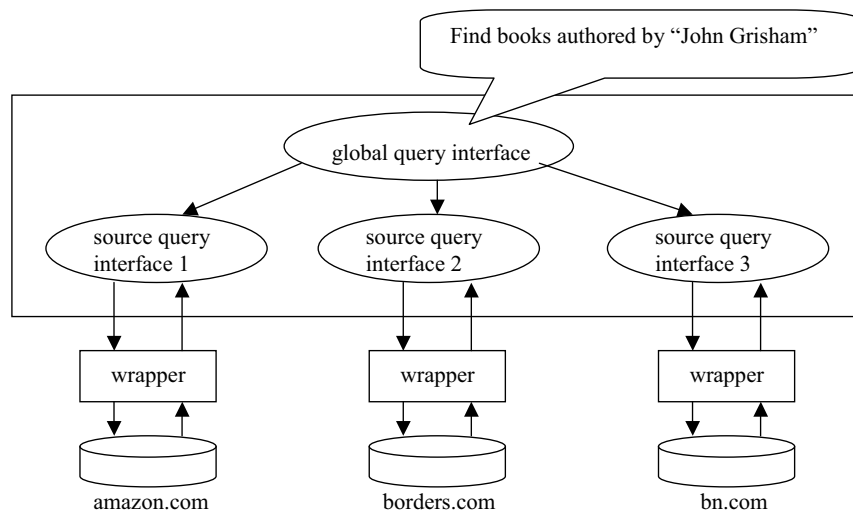


Figure 1.1: An information integration system in the bookstore domain

In Fig 1.1, given a query that is posed over the global query interface (also called the mediated schema), the system translates the query into queries over the different source query interfaces (source schemas). The results are then combined and returned to the user.

1.2 Common Problems

Listed below are some of the common challenges involved in building an information integration system. In later chapters we shall present a real world example of an information integration system we built that addresses some of these challenges.

- Deciding on a Mediated Schema

The schema of the global query interface is often referred to as the mediated schema. The mediated schema is a virtual schema that should capture the salient aspects of the domain. Deciding on a mediated schema is not necessarily an easy task even for simple domains. For example, in the book domain, we might decide to allow users to be able to query on the following attributes of a book: title, publisher, isbn, authors first and last name. These attributes would constitute the mediated schema. What about the following attributes: edition, year published, country published in, category/genre, editor. Whilst these attributes might not be as popular as those listed earlier some users might still find it useful to be able to pose queries on them. However including all these attributes in the mediated schema might overwhelm the user. What constitutes the mediated schema is also dictated by what attributes are commonly available in the source schema. For example amazon.com now allows users to pose keyword queries on the contents of a book, however most other book sources lack this feature, thus including a keyword attribute in the mediated schema might not be practical since it is used by very few sources. Ultimately deciding on a mediated schema is a subjective process and it is the job of experts in the domain to determine what attributes should be included in the mediated schema.

- Source discovery

The goal of an information integration system is to build a uniform interface over several data sources. The more data sources there are the better the coverage of user queries. In some instances, a survey might have been carried out or there might be some community site that will list a lot of sources for a particular domain but this is not often the case. Thus finding data sources will either have to be done manually,

or by implementing some sort of crawler that crawls the web and a classifier to select some of those websites as data-sources for the system (i.e. a focused crawler).

- Finding Semantic Mappings

The acquisition of semantic mappings is a key obstacle in deploying information integration systems. Semantic mappings specify the relationship between the attributes of the mediated schema and the schema of a local source.

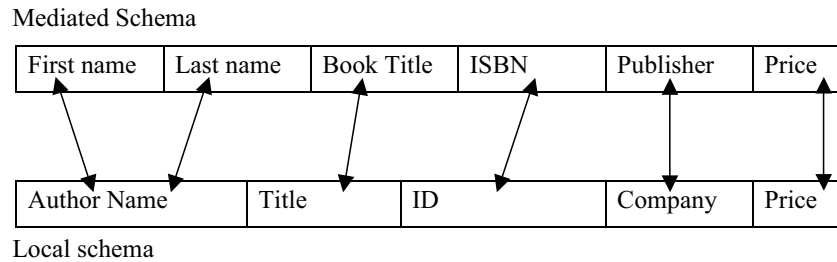


Figure 1.2: Mappings between a local schema and a mediated schema

In figure 1.2, an example of a mapping would be first name and last name in the mediated schema maps to author name in the schema of some local data source.

- Wrapper Construction and Maintenance

Programs which are referred to as wrappers are attached to each data source to handle data extraction, formatting and transformations between the data-model of each source and the data-model of the integration system. For example amazon.com has several forms on its site such as forms that allow a user to subscribe to a mailing list, search for cd's and search for books. It is the responsibility of the wrapper program to know which of these forms to extract, also once the particular form has been found the layout of the form elements will be different from those of other sites, thus the wrapper would need to know how to extract these elements as well. Also once a query has been posed over a site, the results are returned in different formats by the different sources. Some might return the result in a tabular format, others might return it in a list format, etc. Once again it is the job of the wrapper program for each site to extract the results and transform them into common format used by the global interface. In the past wrappers used to be built manually for each site, but over the past few years there has been a lot of research [8, 24] in automating this process though this is sometimes very error prone due to the fact that the sites can sometimes be very different so it's

quite difficult to come up with an algorithm that catches all the different variations a site can have. Writing wrappers is probably one of the most difficult components to build because once the wrappers have been built their also has to be some sort of maintenance policy because it is very common for the layout of the websites of a data source to change. Thus the wrapper needs to be reconstructed occasionally.

- Join Paths

There are times when a source might not contain all the information needed in the results to be returned to the user. In such an instance it might be necessary to do pose the query over multiple sites simultaneously and then combine the results via a join. For example suppose the mediated schema actually consisted of more than one relation say: *BookItem(title, publisher, isbn, price, author first name, author last name)* and *BookReview(title, review)*. In this case there might be local sources that are just dedicated to book items whilst others might just be dedicated to just book reviews, however when queries are posed over the mediated schema we would want to return the review information for each book in the query result and thus it would be necessary to do a join of both types of sources. Or consider the earlier example where we mentioned that amazon.com is one of the few sites that allows keyword search on the contents of a book. This keyword attribute can be included in the mediated schema and even though other sites don't support it, the system could be designed such that results from amazon are joined with results returned by other sites so that only books conforming to the keyword search are returned. For example suppose a user wanted to find "all books authored by John Grisham with keyword lawyer contained in the book", the first part of the query could be conducted on all sites/data sources, once this result is obtained a join could be performed on the isbn field with the results returned from amazon.com since it's the only one supporting keyword queries. Figuring out which sources and which attributes joins should be performed on is a complicated task that sometimes needs to be handled by the information integration system [30]. Also for other data-models such as a relational database, there are times when complex mappings, which are mappings with more than one attribute involved, are spread across multiple tables within a data-source, figuring out join-paths is also crucial in this instance.

- Object Matching

In performing joins between results returned by two different data sources, once a join-path has been selected, the tuples in the two data sets still have to be matched. Object matching refers to the problem of determining whether two tuples refer to the same

object. Consider the previous example on a source dedicated to just book reviews, let's assume the schema of this source is given by: book title and book review. To perform a join with results returned by other data sources the join path will have to be on book title. A book might be listed as "Harry porter and the order of the phoenix" on some data-source and listed as "Order of the phoenix" on another data-source, the object matching problem simply refers to knowing that these two books refer to the same book.

- Query Optimization

Query execution and optimization is another important component of information integration systems. Given the numerous data sources that the system might be built over, it is sometimes unreasonable to query all the sources and wait for results to be returned by all the sources before results are returned to the user. Hence as with regular database systems, an information integration system has to come up with a plan for executing queries taking into account factors such as cost of executing a query and coverage of the results returned. Optimization for information integration systems is usually harder than for regular relational database systems because often the sources are autonomous and thus this makes it hard to evaluate the cost or obtain statistics about the different sources [26].

Numerous works have been conducted, both in the database and artificial intelligence communities, on building information integration systems that address the different challenges listed above [9, 12, 18, 19, 22, 23, 27]. In this paper we discuss some parts of an approach called mass collaboration, which leverages collaborative user-input in addition to existing methodologies to build information integration systems.

1.3 Thesis Overview

This thesis is divided into two sections. In the first section we give a brief overview of *MOBS*, a mass collaboration framework for building information integration systems using input from numerous users over the web. After which we discuss a real-world information integration system built using *MOBS*, in particular we discuss the query translation module for this system. In the second section we discuss monitoring systems, which detect changes on websites. We also present an example of such a system, a monitoring system we built over two different domains. Lastly we point out the similarities between building a monitoring system and an information integration system and discuss monitoring as a variant of information integration.

The major contribution of this thesis is that it presents and discusses building a query translation module for information integration systems using the *MOBS* approach. It also presents a monitoring system that was built and discusses monitoring in the context of information integration. Also using the systems that were built as examples, this thesis highlights what is involved in building such systems and also discusses the lessons that were learnt in building these systems and possible directions for future work.

Chapter 2

Mass Collaboration to Build Systems (MOBS)

2.1 What is MOBS?

MOBS is an acronym for “Mass Collaboration to Build Systems” [14, 28]. The notion behind *MOBS* is to ask users of an information integration system to “pay” for using the system by answering relatively easy questions. Those answers are then used to further build the system. By doing this a great amount of the burden of building and maintaining information integration system is lifted from system administrators and spread thinly over a mass of users. Mass collaboration does not have to replace existing techniques such as machine learning but rather it complements these techniques and helps amplifies and improve their results. Also mass collaboration is applicable in a wide variety of settings, it can be used by employees within an organization to integrate data sources within the organization, in scientific communities such as in bio-informatics, or asking members of a web community such as craigslist.org (which is a site dedicated to helping people in a particular local region find jobs, housing, roommates, etc) to help expand and maintain the site. Mass collaboration has the potential to drastically reduce the cost of building information integration systems and spread it’s use in a variety of domains but it also raises a lot of challenges such as soliciting and combining user feedback. The *MOBS* approach to handling such challenges is discussed in the following sections.

2.2 Framework

The MOBS framework is described below using as an example the problem of identifying query interfaces. Suppose that an information integration system is to be built over bookstore

sources, the html pages of a URL of such sources usually contain many input forms. Some of these forms are indeed query interfaces to bookstores but others are not, e.g., forms eliciting user's contact information for promotions and mailing lists. That is given a form F of a page, say amazon.com, a crucial problem is recognizing that F is indeed a bookstore query interface[11]. This is part of the wrapper construction problem mention in the 1st chapter, here we're trying to build as part of the wrapper for each source a means of extracting only the forms that pertain to book merchandize. The *MOBS* approach to solving this problem is described below.

1. Decomposing the problem: To apply *MOBS* to any particular problem, the problem is first broken down into a series of yes/no questions such that knowing the answers to these questions amounts to solving the problem. The series of questions can be viewed as a set of parameters $P = P_1, P_2, P_3$ whose values must be determined. The task therefore would be to obtain user feedback to help determine the values of P . As users interact with the system, they'll provide feedback and periodically the system will combine all the user feedback that has been accumulated so far to arrive at a new parameter configuration P' , the assumption is that with sufficient user feedback the system will converge to the right parameter configuration. For example the problem of recognizing a particular form can be broken down into a single parameter/question: *'Is the form F a query interface to a bookstore?'*
2. Soliciting user answers: When a user queries the system (e.g. find all books with the title "things fall apart") the user is first made to pay for the query result by having the user answer a question first. The question is displayed with the form highlighted; the intuition is that since the html pages of a data source are designed for human consumption it should be relatively easy for the user to answer the yes/no question. Only after the user has answered are the results of the query displayed. The frequency of questions asked is a parameter that can be adjusted by the system, e.g. one question per query or one question per day.
3. Assessing users' reliability: One of the concerns with using the *MOBS* approach is having malicious users use the system. How does the system protect itself from users that intentionally seek out to give false answers or perhaps an ignorant user that isn't able to properly answer questions? To handle these scenarios the system retains a set of evaluation questions to which the system already knows the answers. These questions are occasionally posed to users' and a weight is computed for each user that serves as a measure of the quality of their feedback. Before a user has answered a certain number

of questions (pre-specified threshold) their weights are set to zero, once they have answered enough questions their weight is set to the fraction of evaluation questions they answered correctly. Once a user's weight has been computed, the evaluating process is halted. Users with a weight above a certain threshold are considered trustworthy, and theirs is the only input that is used to help converge the system to the correct configuration parameters. So as not to be quite predictable, the system mixes evaluation questions with teaching questions (those used to get feedback on unverified query interfaces).

4. Combining user answers: As users query the system, the different parameters/questions receive a steady stream of yes/no answers. The stream is monitored and terminated, i.e. user feedback is no longer requested, once a stopping condition is reached. Suppose that for a particular parameter/question there are a total of N answers from trustworthy users, a stopping condition could be:
 - (a) The number of majority answers (yes or no) reaches $N \times 0.65$ and N exceeds t_1 or
 - (b) N exceeds t_2 and t_1 less than t_2 are given or
 - (c) Techniques from prior work on combining results from multiple learners can be applied [13, 29], where in this case the users are considered to be learners for the system.

Once stopped, the majority answer is assigned as the final value of the question/parameter.

2.3 Incentive Models

The key element of the MOBS approach is user feedback. The more users there are and the more they interact with the system and answers questions the faster the various parameters will converge to their correct configuration parameters. Listed below are some of the different ways of getting the user to interact with the system and provide the feedback the system needs.

1. Volunteer feedback with delayed gratification: Often users will be willing to provide feedback to a system if they know that doing so would reap some benefit in the future. For example scientist might be willing to answer questions to help improve an information integration system over data pertaining to the human genome. In this instance

the reward of being able to pose richer queries over the data set in the future would probably be enough to entice users into taking the time to provide feedback to the system.

2. Volunteer feedback with instant gratification: Another model that can be used is one that provides additional features to user's who provide feedback to the system. For example assuming an information integration system has already discovered the semantic mappings for Author and Title but not ISBN for most of its local data sources, such a system might allow users willing to answer question about ISBN to query on all attributes for which the mappings are already known, in this case Author and Title. If a user elects not to provide feedback the system would allow the user to query on only a subset of the mappings found so far, e.g. just Title. This way users would be enticed to provide feedback to the system because of the instant gratification of additional features. A real world example of this is the google toolbar. In order to use additional features of the toolbar such as seeing the page rank of a page on the web, the user has to consent to having google collect implicit feedback from their computer.
3. Monopoly/better service model: In this model the user is forced to provide feedback and thus have no choice. Consider the following example. Most cities have a government run company, e.g. the Chicago transit authority (CTA), in charge of public transportation such as buses. A lot of these companies have websites with query interfaces where users can query for information about different bus routes and schedule. Suppose the federal government wanted to build an information integration system over all these sites, it could decide that in order to use any of the local sites users would have to answer a few questions before being given access to the site. Since in each city, only the government has information on all the routes, the government has a monopoly and is therefore able to force its internet users to provide this information. Another example would be a company intranet; employees could be forced to answer questions (which will help integrate the company's different data sources) before being allowed to access pages on the intranet.
4. Helper application model: If an application does not have a monopoly or better service model it may still be able to obtain feedback forcefully from users by employing not it's own target users but rather users of a system that has a monopoly or provides better service than any other like system. Returning to the previous public transportation example suppose the governments plan backfires and users stop using the websites since they can still access the information they need on paper print versions of the routes and

schedule. The government may decide to elicit information from a different set of users, e.g. users filing their tax information over the web. In this case the tax website would be the helper application. In fact the government could use these set of users to build all sorts of systems since it is unlikely that people would give up the ease of filing taxes online. Returning to the second example of the company intranet, a company could also employ users of its own intranet to help a client build an information integration system over the client's data-sources.

Whatever the model may be, a principle that should be followed is that users who pay more must be able to get higher quality service from the system. Also even if forced feedback is being used, the system should not overwhelm the user with too many requests for input. Also if the system is already established with a stable user population and we simply want to expand its capabilities then any one of the four models can be used. On the other hand if we're building an information integration system from the ground up, then the monopoly/better service model cannot be used but any of the other three models may be used.

Chapter 3

Applying MOBS to an II System over a Book Domain

3.1 System Overview

In this chapter we discuss as an example, an information integration system built over bookstore domain using MOBS and machine learning, in particular we discuss the query translation module for this system [4]. The book domain was chosen because it is easy to understand and is well known in information integration literature [11].

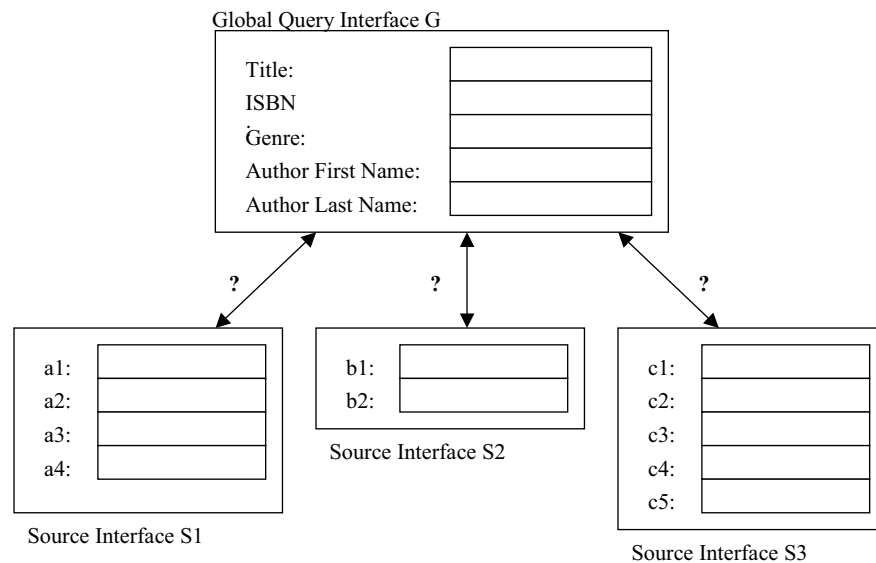


Figure 3.1: Using MOBS to match the global interface and the source interfaces

A simple mediated schema was settled upon for the global query interface. The schema

had four attributes: title of book, genre, ISBN, author's name. An information integration system was then built on seventeen sources pertaining to books, in figure 3.1 just three of the sources are shown.

Initially the system does not know how to find the mappings between the global query interface and the local query interfaces of the different sources. MOBS is applied to the system to help it find the mappings, also the initial problem of finding the different sources that the system is built upon was solved using MOBS as well. Building the system can be summarized into three stages

1. Source discovery: finding the query interfaces
2. Matching query interfaces
3. Translating queries from the global interface to the local interfaces

The first two stages are described briefly below, whilst the last is described in more detail.

3.2 Source Discovery and Recognizing Query Interfaces

One of the challenges in building the global query interface, is finding numerous sources to build it on top of. The more sources there are the better the coverage of query results returned to users. To solve this problem we applied MOBS on top of current machine learning techniques, in particular TF/IDF classification method. TF/IDF was applied to a pre-collected repository of websites with query interfaces [11]. 24 of these sources were identified as being related to books by TF/IDF. MOBS was then applied to help improve the accuracy of these results.

Since the system was being built from scratch the helper application incentive model was used. The assignments section of a course home page was used to elicit feedback from students seeking to download assignments from the page. Students were shown html pages with possibly multiple query forms and were asked to identify whether a highlighted form was a query interface to a bookstore. Within a few days the students had helped the system converge to the correct configuration parameters with 100 % accuracy. Out of the 24 forms identified positively by TF/IDF, 17 were found to be actual query interfaces to bookstores.

3.3 Matching Attributes of Interfaces

For each of the seventeen identified book source interfaces, the next step was to match its input fields with those of the global interface. Consider figure 3.1, if the system wanted to match input field b1 of the interface of source s2 the problem would be decomposed into four questions. “b1=title?”, “b1=genre?”, “b1=author?” and “b1=ISBN?”. The same MOBS mechanism on the course homepage is the used to elicit feedback from its users in answering these questions. Figure 3.2 shows how a sample question is presented to the user.

Is the highlighted field used to search on TITLE?

YES NO

b1:

b2:

Figure 3.2: A sample MOBS question

The form is shown with an input field highlighted and the system poses a question about this input field to the user. If the answer for a question converges to yes, matching is stopped for that input field and the match encoded by the question is returned.

3.4 Translating User Queries

Once the input fields of the query interface of a source has been matched with the attributes of the global query interface, that source is now added to the pool of sources queried when users pose a query over the global interface. For example assuming the user enters “Romeo and Juliet” in the title field of the global query interface, how does this then get filled into the different interfaces of the local sources. Doing this is not necessarily as simple as just locating which field matches to title in the local source and filling in the query in this field. This is so because it is possible for two fields within the local source to have been matched with the title attribute. For example consider the example of source S2; both fields b1 and b2 could both be used to search on title. B1 could be a dropdown list containing the options search by title, and search by author, whilst b2 could be a text box for inputting the query. Also in the case of the author attribute, the global interface actually has two fields for this attribute, one for the first name and another for the last name. The interface of a local

source might have just one text field for entering the author name, this field would have been matched to the author attribute by MOBS but it is up to the translation module to figure out how to combine the first name and last name entered in the global interface into this one field in the interface of the local source. The translation module achieves these tasks using a rule-based approach. An example of a rule for the previous two examples would be “if title matches a text field and dropdown list in the local source, fill in the query in the text field and choose the default for the drop-down list” and “if author matches a single text field in the local source, enter the concatenation of the first name and last name from the global interface into the text field”. Thus the translation module can be viewed as working in two stages:

- Coming up with the rules
- Using those rules to translate queries over the global interface to the local sources

3.4.1 Rules Generation

Once the MOBS mechanism has figured out which attributes of the mediated schema the different fields of a local interface form match with, the query translation module is called to generate rules for this interface. For example in figure 3.3, the MOBS mechanism has been applied to the two data sources S1 and S2 and their respective input fields have converged to the correct attributes of the global interface that they match with. At this stage the

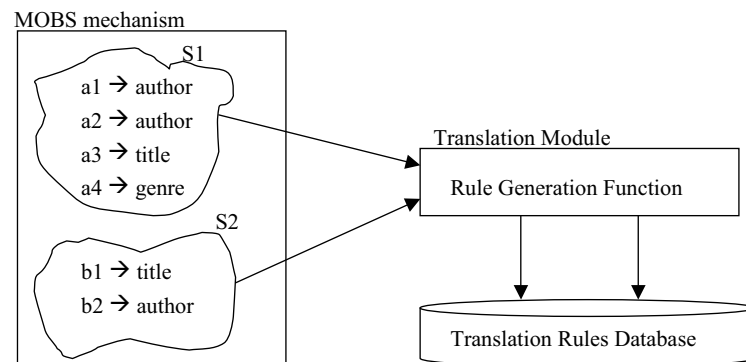


Figure 3.3: Generating translation rules

rules generation function of the translation module is now called upon to come up with rules for translating queries posed over the global interface unto the interfaces of S1 and S2. For each source, the function receives as an argument the input fields of the source and the attributes of the global schema they match with. The rule generation function contains a

list of manually specified rules on how to match input fields for each attribute of the global schema. That is there is a separate rule list for each of the global schema attributes: title, genre, ISBN and author. In figure 3.3 the input fields a1 and a2 of source S1 match with the author attribute. The translation module would then search its author's rule list for the first applicable rule. An example of an applicable rule in this scenario would be: *“If there are two input fields matched with the author attribute, write the first name into the first field and the last name into the second field (where first name and last name are values obtained from the query posed over the global interface)”*. The text in italics is the rule's precondition, i.e. this is how the system determines whether a rule is applicable or not. Once a rule has been found, the rule is stored in the translation database so that later on when a local source needs to be queried (discussed in the next section) the system does not need to search for which rules to use and can simply look up which rules are applicable to the input fields of a source in the translation database. The rule generation function can also be called repeatedly for the same source, when this happens the function looks for the next applicable rule. Returning to the previous example, if the rule generation function were called again for the input fields of source S1 pertaining to the author attribute field, the function would look for the next applicable rule skipping the earlier found rule. For example in this case it could try reversing which fields the first names and last names are entered into and thus come up with the rule: “write the first name into the second field and the last name into the first field”.

The rule list was specified manually based on heuristics, which seemed intuitive to us. This intuition was from surveying 3 of the 17 bookstore forms in the system and trying to come up with generic rules that would help the system translate queries over the global interface onto the interfaces of the 3 bookstore sources. We did not survey all seventeen sources for the same reason testing and training data are kept separate in machine learning algorithms. That is we did not want to come up with a look up table of rules that would work very well with all seventeen book sources but fail miserably for any new bookstore source added. Shown below is a list with some examples of rules that would be applicable to the different input fields of source S1 and S2.

Example rule list:

1. If there are two text fields in the source matching with author attribute, enter the first name in the first field and the last name in the second field
2. If there is just one text field in the source matching with author fill the last name in that field

3. If there is just one text field in the source matching with author fill the concatenation of the first name and last name in that field
4. If there are one or more text fields matched with the title attribute fill the title in the first field
5. If there is one text field and a dropdown list matched with the title attribute, fill the title in the text field and choose the default option in the drop down list
6. If there is a dropdown list matched with the genre attribute and if any of the options match the text entered in the global interface for genre select that option
7. If there is a dropdown list matched with genre attribute, choose the default option in the drop down list

3.4.2 Using Rules to Translate Queries

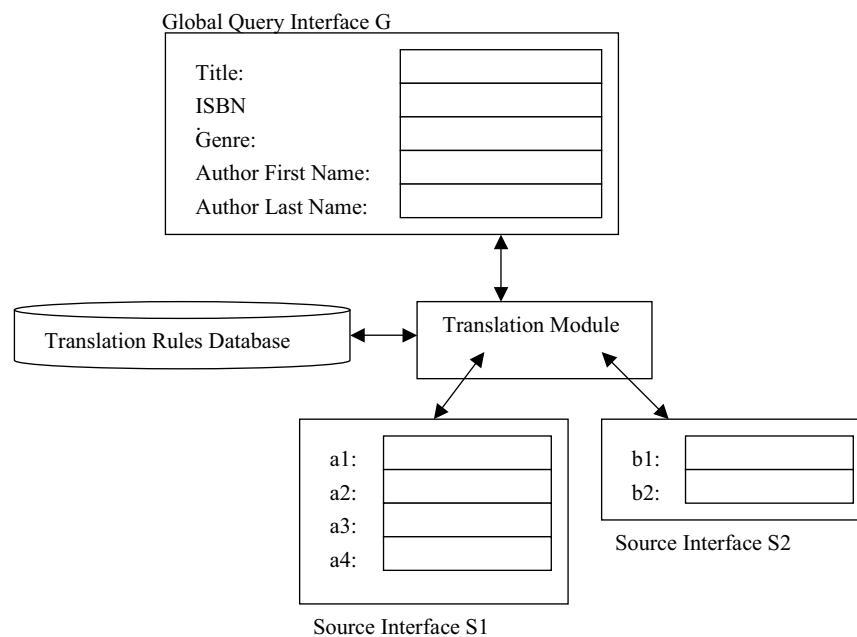


Figure 3.4: Translating queries with rules

Once the MOBS mechanism has been used to match the input fields of a source to the appropriate attributes in the global query interface, the translation module is used to generate rules, which are then stored in a database. Later on when a user poses a query

over the global query interface, the interface calls upon the translation module to translate that query over the local interfaces of the different sources in the system. The translation module then looks up the database for previously saved rules on how to query the different sources.

Deep-Web Book Searcher

Title of Book	Harry porter
Author FirstName	
Author LastName	
Genre	
ISBN	

Sources Currently Queried by System: 18

Figure 3.5: Queries posed over global interface are translated

When a rule is stored in the database by the rule generation function of the translation module, an id that uniquely identifies the source and the attribute of the global interface this rule applies to are stored with the rule. Thus when a user poses a query and the translation module is called upon it can quickly lookup the source id in the database and then look up the attribute the query is on to see if there's a rule for translating it unto the source. For example assuming the user enters the "John" and "Grisham" into the author first name and author last name of the global interface in order to find all books authored by "John Grisham", in order to query the first source S1 the translation module will look in its rule database for a previously saved rule: "Select rule from translation table where source=s1 and attribute=author". At this point a previously saved rule such as "write first name into first field and last name into second field" is returned. Also when rules are stored in the database, the exact location of the input field within the form (specified by some offset) is store with the rule as well so that when queries are translated via rules the system can easily determine where first field and second field are.

Sometimes a query is entered on the global interface that a local source cannot perform, in such an instance the source is simply skipped. For example if a user entered "science fiction" in the genre field on the global query interface, source S1 would be queried by the system not source S2 (going by the attributes matched by the MOBS mechanism in figure 3.3). For source S2, the translation module would try and look up a previously saved rule in its

database: “Select rule from translation table where source=S2 and attribute=genre”, but since none of the input fields of source S2 were matched to the genre attribute by the MOBS mechanism, no rule for the genre attribute would have been generated by the rule generation function. Thus on querying the translation database, the result returned would be empty at which point the translation module would simply skip source S2 for this particular query. Thus sources that cannot satisfy queries posed over the mediated schema are actually never visited by the system during query translation and execution.

3.4.3 Combining and Returning Results of Queries

Book Search Results

Source 1 (Source unavailable): <http://www.1bookstreet.com/search.asp?mscssid=6A82ASPF476X9L2XJJ9TUU7>
 o [Submitted Form](#)
 o [Results](#) (Source Unavailable)
 Source 2 : http://www.albooks.com/cgi-bin/alSearch?act=advSearch&WVSESSION_ID=130629225
 o [Submitted Form](#)
 o [Results](#)
 Source 4 : <http://dogbert.abebooks.com/abep/il.dll?ph=1>
 o [Submitted Form](#)
 o [Results](#) (Source Unavailable)

Figure 3.6: Results page

After entering a query on the global interface, the system queries all the sources that can satisfy this query via the translation module. A results page is then generated by the system with links to the query results from the different local sources that were queried. In addition to links to the query results, there is also a link to the form submitted for each local source. This form shows the query with the input fields that were queried containing the query values. By showing the filled in form in this manner, it allows the user to visually verify that queries were translated properly by the system.

Advanced Search

You can narrow your search by selecting one or more options below.

ISBN:

Title:

Author:

Price: From To

Figure 3.7: Filled in form on a local source for visually verifying translations

The results returned by queries on the global interface can be viewed simply as a concatenation of the results of querying the different local sources. That is the system never attempts to do any joins on the results returned by any two sources, neither does it try to combine two sources in order to satisfy a query. This is so because the mediated schema is composed of only one relation. There is also an implicit assumption that each source is sufficient on it's own to satisfy queries. As explained earlier when this assumption fails and a source cannot be queried, e.g. user enters in a genre in the global interface but the source doesn't contain any fields matching to genre, the source is simply skipped during translation. In addition in most cases the assumption holds since for the bookstore domain each site maintains its own inventory, had we been trying to incorporate other book domains such as sites dedicated to book reviews then the assumption wouldn't hold because we would need to combine results with bookstore domains in order to supply reviews for books returned in their queries.

One other thing to note is that the translation process can be seen as using the local as view (LAV) [25, 27] methodology as opposed to global as view (GAV)[6, 13]. It can be viewed as LAV because the rules that we come up with for fields of a source are not expressed in conjunction with those of any other sources and is simply posed as a query over the mediated schema relation. However we do not have to bother with complex recursive algorithms to translate queries, a common drawback of using LAV, since the mediated schema is simply composed of a single relation.

3.5 Experimental Results

In summary, we were able to use MOBS mechanism implemented on a course homepage to build an information integration system over the bookstore domain in a short amount of time. Using MOBS the students (users) were able to help the system classify all 24 forms properly, recognizing 17 of them to be query interfaces to bookstores. This process took 5 days. Also within 7 days, the students successfully matched 10 out of the 17 interfaces (65 fields in all) with 97% precision and recall [14, 28]. There was not enough time to converge the remaining sources due to the semester ending in December. Lastly the manually specified rules were able to translate queries properly for all the sources. Since the system was built in December several tests and demos have been run in which queries were entered over the global interface and the system has been able to correctly translate the queries unto the different local interfaces and return the appropriate results.

3.6 Related Work

As far as we know MOBS is the first work to apply a mass collaborative approach to building information integration systems. Mass collaboration has been studied for other applications though such as building tech support websites [33], knowledge bases [34], and user trust networks [7]. Also as mentioned earlier, most information integration systems are built manually and this is very labor intensive. There have been several works on reducing the labor cost of this process which address specific challenges of building an information integration system. Challenges such as schema matching [32] and wrapper construction [8, 24].

The work on MOBS also shares with research on the semantic web. The semantic web is advocated as the next generation World Wide Web where data is marked up and software programs can exploit marked up data to better meet the needs of users. Examples of common issues shared include enticing users to provide feedback and combining information of varying quality [16]. The mass collaboration framework also shares an interesting relationship with machine learning in the sense that each validated user can be thought of as a learner, which has been trained and is ready to make predictions. The mass collaboration framework can also exhibit properties of autonomic systems [2] such as self-healing and self-improving. The difference being that traditional autonomic systems observe the environment and then tune the system appropriately, here the environment (users) observes the system and then tunes it appropriately. *MOBS* in general differs from previous work because it provides a systematic effort to address cost reduction for the information integration process as a whole.

3.7 Summary

The current cost of ownership of information integration systems is high due to the need to manually build such systems. In this chapter we presented how mass collaboration approach was successfully applied to build a cost-effective information integration system over book sources and also described in particular the modules that allow for successfully querying the system.

Chapter 4

Building a Monitoring System

4.1 Introduction

In today's world, one of the most common sources of information is the web. Looking for directions, best prices on hotels, a particular book, reviews on a recently released movie, these are just a few examples of information easily obtainable from the web. The actual amount of information accessible over the web is quite large, for instance the popular search engine, google, which crawls only a fraction of the web claims to index over 4.2 billion pages. A single user will only be interested in a minute fraction of these pages, and often times the user will be interested in what has recently been modified on these pages.

Consider as an example an avid sports fan who likes basketball and who supports the Los Angeles Lakers. One can imagine such a fan combing the numerous websites dedicated to sports looking for new articles pertaining to the Los Angeles Lakers on a daily basis. Another example would be in searching for a job. Most companies today that have a website have a section of the site dedicated to listing available job openings. One can again imagine an unemployed individual checking such sites occasionally for new job openings. Lastly consider an example of a trader in the financial industry with a portfolio focused on a select group of companies. One can imagine that the trader would be interested not only in the changes in the stock prices of such companies but would also be interested in when new documents such as quarterly reports appear on the websites of these companies, or of any mention of such companies in headlines on news websites.

With all of the above examples users have to regularly monitor some websites for new information they consider interesting. In this chapter we discuss building systems to automatically monitor websites for changes. Based on a user's preferences the system will notify a user when there are new changes on the pages of a website the user is interested in. In particular we present the architecture of a real-world system we built [3] to monitor websites

for changes in two different domains. We also discuss challenges of improving the results of the system and suggest direction for future work to address these challenges. Viewing monitoring as a special application of data-integration is also discussed.

4.2 The Two Domains

We chose two different domains to monitor for changes. The first being websites of prominent researchers in the field of databases. The list of researchers was determined from the program committees' of past SIGMOD conferences [5]. The homepages of these researchers were monitored for modifications in the publications available for download on their homepages, i.e. the system is able to determine when a new publication has either been added to or removed from a researcher's homepage. The second domain was over websites of courses at the University of Illinois. Here the system simply detects when anything has changed on a page on the website and reports to the user which page in particular the change occurred on.

4.3 Overview of the Monitoring System

The general idea behind our monitoring system is that given a list of sites, we want to be able to take periodical snapshots of the sites. By taking a snapshot of a website we mean that we cache all of the information that we are interested in within that website. We can achieve this by simply caching all the data in the website or we could extract the data we find interesting first and then do the caching, we discuss this further in a later section. Once we are able to take snapshots of all the websites on a regular basis, we can monitor the sites for changes by comparing snapshots taken at different times and reporting the differences found between them. Figure 4.1 shows the different modules in the monitoring system, each of these modules are described in more detail in the next section. An arrow leading from a module A to another module B in the figure means that A invokes B in some manner. Either Module A executes Module B or it reads or write data to B. In the latter cases Module B would have to be one of the two data storing modules in the system (represented in the figure with cylindrical shapes).

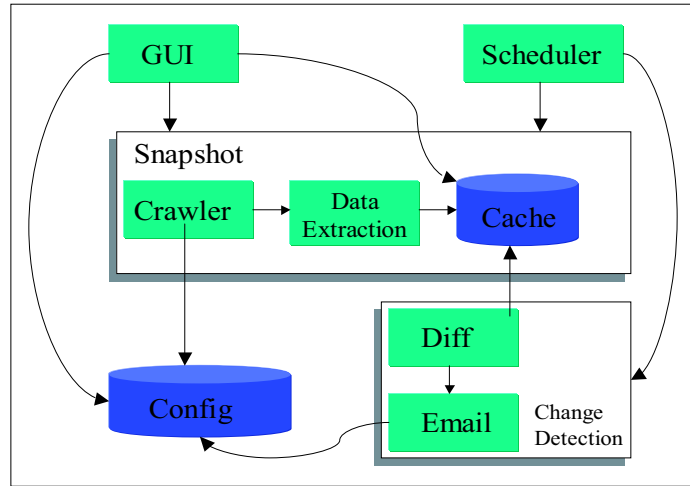


Figure 4.1: Monitoring System Modules

4.4 Description of Modules

4.4.1 GUI

A graphical user interface (GUI) is provided which serves as the front end to the system. It provides direct access to all the features of the monitoring system that are provided by the other modules. For example, administrator's can manually force crawling and caching (snapshots) of any of the monitored sites via this interface, they can add new researchers or courses to the system for monitoring. Regular users can select any number of sites from the list of sites being monitored by the system and then subscribe to receive reports via email on changes within these sites, they can also view cached copies of websites and conduct queries via GUI elements to see reports on what has changed between any two different versions of cached websites. As we discuss the other modules in the monitoring system we shall also discuss how the GUI interacts with these modules. Figure 4.1 is a screenshot of the GUI showing the front page of the system.

4.4.2 Configuration

A batch process was used to specify an initial set of websites to be monitored by the system. However if an Administrator encounters a new website they would like to add to the system for monitoring, a form is provided in the GUI that allows them to add additional websites to the system. Forms are also provided to allow a particular user to be able to specify which of these sites they are interested in monitoring. Figure 4.5 shows the form for adding the

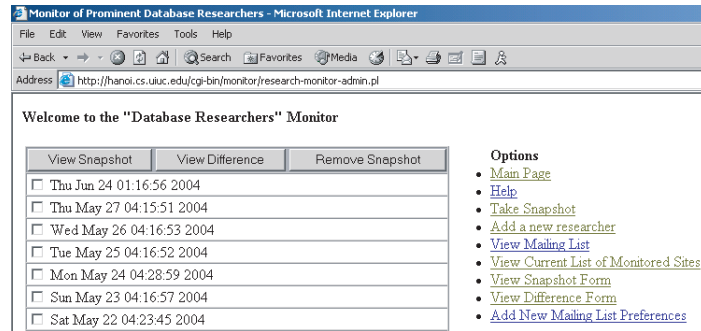


Figure 4.2: Main page of monitoring system

website of a researcher to the system for monitoring. All of this information is managed by the configuration module which stores the data in xml files. We decided on using xml files to store this data, because it would allow for us to build a datasets that could be easily shared with other applications in the future. The config module manages the reading and writing of these xml files which contain configuration details for different modules in the system, mainly the crawler and the email notification modules. There are three xml files. Two of

```
<!DOCTYPE Researchers [
  <!ELEMENT Researchers (Researcher+)>
  <!ELEMENT Researcher (Name, Email, Site+)>
  <!ELEMENT Site (URL,Depth?, Allow?)>
  <!ELEMENT Name (#PCDATA)>
  <!ELEMENT Email(#PCDATA)>
  <!ELEMENT URL (#PCDATA)>
  <!ELEMENT Depth (#PCDATA)>
  <!ELEMENT Allow (#PCDATA)>
]>
```

Figure 4.3: listing of dtd for researchers

the files are domain dependent and specify configuration details for the crawler. One is a file containing details about the different researchers being monitored, and the other contains details about the different courses being monitored. Both files in essence contain the list of URL's to be crawled and monitored by the system. The third file contains details about which user's have subscribed to the system and which sites they are monitoring, this is used primarily by the email notification module. Associated with each researcher being monitored is the name of the researcher, the researcher's email (used by the system only to uniquely identify researchers), and a list of the researcher's websites/homepages. For each website a depth and allow parameter may be specified. The depth parameter indicates how deep

```

<Researcher>
  <Email>alevy@cs.wash.edu</email>
  <Name>Alon Y.Halevy</name>
  <Site>
    <url>http://data.cs.washington.edu/</url>
    <allow>cs.washington.edu</allow>
    <depth>2</depth>
  </Site>
  <Site>
    <url>http://www.cs.washington.edu/homes/alon/</url>
    <allow>http://www.cs.washington.edu/homes/alon/</allow>
  </Site>
  <Site>
    <url>http://www.cs.washington.edu/research/semweb/</url>
    <allow>http://www.cs.washington.edu/research/semweb/</allow>
  </Site>
</Researcher>

```

Figure 4.4: Sample xml generated by config module after adding a new researcher

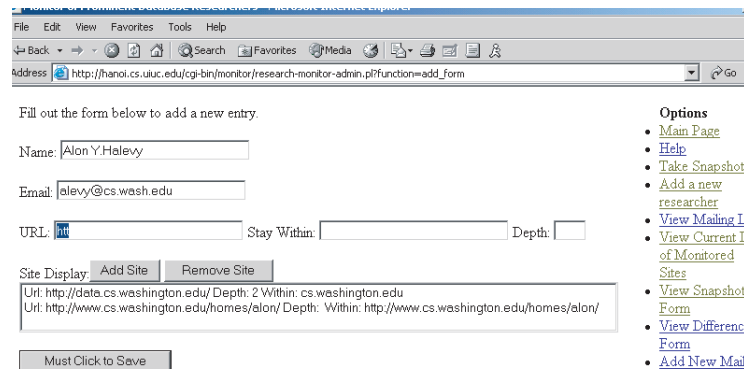
the crawler should crawl within this website and the allow parameter specifies a string that all links the crawler follows must contain, these allows restricting which links are crawled within a website to a particular domain. Figure 4.3 and figure 4.4 show the dtd and sample xml for researchers. The dtd for the Courses is similar to that of the Researchers, the only difference being that instead of name and email elements as with the researchers dtd, there are description and id elements. There is also a dtd for the users subscribing to the system. The users xml file contains the emails of subscribed users, and which sites they've chosen to monitor.

4.4.3 Taking Snapshots

All of the tasks that need to be performed periodically by the monitoring system is handled by the scheduler. One of the tasks is that of taking snapshots of all the sites in the system regularly. Administrators can also force snapshots to be taken via the GUI if needed. Once a snapshot has been taken, users can view the contents of the snapshot via the GUI and can also compare any two snapshots for differences. Taking a snapshot of a website comprises of crawling the website, extracting the data within the site and lastly caching this data. These three tasks are handled by separate modules which are described in more detail below.

4.4.4 Crawler

For both courses and researchers, multiple sites can be specified for any particular researcher or course. Thus it's possible to simply specify the exact url of the pages that need to be monitored for any particular course or researcher as shown in the figure 4.5. This is not why



Fill out the form below to add a new entry.

Name:

Email:

URL: Stay Within: Depth:

Site Display:

Url:
Url:

Options

- [Main Page](#)
- [Help](#)
- [Take Snapshot](#)
- [Add a new researcher](#)
- [View Mailing Li](#)
- [View Current Li](#)
- [of Monitored Sites](#)
- [View Snapshot Form](#)
- [View Difference Form](#)
- [Add New Maili](#)

Figure 4.5: Form for entering new researchers, similar forms exist for users and courses

specifying multiple url's are allowed though as this would be tedious for the user, the whole purpose of this is to allow for a researcher or course that has more than one homepage. Not only did we want a user to be able to specify that a particular page on the web be monitored by the system, we wanted to user to be able to specify that a whole website be monitored by the system and thereby not have to worry about which particular page on the site a change is likely to occur on. To achieve this a crawler was implemented as one of the modules of the system. It is sufficient to simply specify a homepage for a researcher/course, and it will be crawled by the system. The crawl of a website is guided by the depth and allow parameters associated with each website. These parameters are obtained from the configuration module which manages information on which sites to crawl on how to crawl them via xml files as shown in figure 4.3 and figure 4.4. A depth of 0 indicates that a site shouldn't be crawled. A depth greater than 0, will cause the page to be crawled, with the value of depth determining when the crawl will stop. For example specifying depth 1 for a website indicates that all links on the homepage will be explored during the crawl, but the links on the pages they lead to will not be explored. Not specifying a depth for a website indicates infinite depth, i.e. the system will keep crawling until it runs out of links to follow. Another way to limit the scope of a crawl is via the allow parameter associated with each website. This parameter allows the user to specify what type of links are to be explored whilst crawling a website, for example given the website www.cs.uiuc.edu, the user might want only links within the uiuc.edu domain to be explored, this would be specified via the

allow parameter.

4.4.5 Data Extraction

As each page is fetched during the crawl process, it needs to be cached. Before or after caching a page though, the data that needs to be monitored has to be extracted. What kind of data gets extracted is of course very much dependent on the domain. For example if websites are being monitored for financial information, then the kind of data extracted might include stock prices. Our monitoring system is quite modular and it allows for an



Figure 4.6: Users can view snapshots, which shows data that was extracted from a researcher's site and then cached

administrator to write different data extraction modules for different application domains. We monitor two domains at this time, researchers' and courses' websites. For courses no data is being extracted because all we're monitoring is whether any content has changed within a course website. For researchers we're interested in monitoring a particular type of content (downloadable publications) within their website so a data extraction module is provided. For each researchers website we extract the exact url and some textual description for all postscript, pdf or powerpoint files (this is a parameter that can be specified within the module). Currently the module picks up a lot of noise in trying to extract publications from a researcher's site, examples of such noise includes postscripts files containing course schedules, class assignments, etc. At the moment we simply use a set of manual rules to try and filter out such noise. An example of such a rule is: exclude all postscript files found on pages with a title or description that contains any of the words: course, fall, spring,

summer and exams. The manual rules do a decent job but the final publications list that is extracted still contains some noise, we do not worry about that at this point because the main focus of the project is in building a monitoring system and not on data-extraction. For our monitoring system, caching of pages fetched during the crawl process is done after the data is extracted, i.e. only the extracted data gets cached. This is not necessary, we could have chosen to cache the entire contents of each web page and then data extraction would have been deferred until when the system needs to figure out what has changed. We chose to extract the data first simply because it helps save some space, i.e. why cache any data that isn't needed by the application.

4.4.6 Caching

Caching is an integral part of the monitoring system. In order to detect what has changed on a website, the current contents of the website needs to be compared with it's prior contents. The particular content that is compared is determined by which data is extracted, i.e. what we want to monitor. But irrespective of what data is extracted from the website, the prior contents of the website have to be saved either before or after data is extracted. For our system, the data-extraction module for the domain in question is run first and then that data is cached. How often data gets cached is determined by the scheduler. The scheduler implements two different types of caching policies.

1. Archive policy: With this type of caching policy the data is cached frequently at regular intervals, irrespective of whether there is a change or not. Each time caching is performed; the data for all the sites is stored all over again with a timestamp. For example for the researchers website, once the list of publications has been extracted, it is cached on a daily basis. Thus the system has a record of which publications were found within a website on any given day. With the archive policy the user can query the archive to see for instance what has changed between any two days by comparing the cached data for both days. This is the advantage of using an archive policy, it keeps a history of the contents of a site, and this allows the user to be able to query the system for what changed for any period in its history. The disadvantage of using this kind of policy is that maintaining a history can be expensive in terms of hard disk space. All the data has to be stored each time caching is done, if numerous websites are to be monitored frequently a lot of space might be needed.
2. ii. Overwrite policy: With this type of caching policy, the data is cached only when it changes. Thus there is only once cached copy of all the websites being monitored and

the cache is updated only when there's a change. The contents of the course websites being monitored are cached using this policy. Thus each time pages are fetched during a crawl, the contents of the page are only stored on hard-disk if there is a difference between the page that was just fetched and the page currently cached on disk that was fetched at some earlier point in time. The advantage of using this policy is it wastes less space, but the disadvantage is now there's no history since cached data gets overwritten whenever there's a change.

4.4.7 Finding Changes

In order to be able to report that a change has occurred the monitoring system needs to be able to compare the current contents of the website with a previously cached copy. The diff module achieves this comparison.

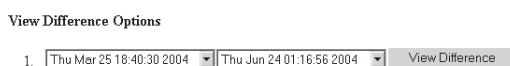


Figure 4.7: Form to allow users to select which time-stamped snapshots to compare

When an archive caching policy is in effect, the diff module also enables two copies of the websites being monitored that were cached at two different times to be compared. The Implementation of the diff module is also very specific to the domain in question as with the data-extraction module. Since the data that is extracted will be different for different domains, comparing the extracted data will be different for different domains as well. Also just as the data-extraction module determines what constitutes interesting content on a website, the diff module determines what constitutes an interesting change. Returning to the example of monitoring websites for some financial application, a data-extraction module for this domain might specify that stock prices be extracted, that is the interesting content. A diff module might specify that a change be reported whenever there's any change in between current stock prices extracted from the website and that extracted from a cached copy, or it might specify additional constraints such as report a change only when the price goes up or down by more than 5%.

A diff module is provided for the two domains monitored by our system. For researchers websites a difference is reported whenever a downloadable publication with a unique url is either removed or added to a researcher's website. That is if there are duplicate links to the same downloadable publications within a website, changes to the number of duplicates, i.e. removing/adding one of the duplicates will not be reported. For the courses website a difference is reported whenever the characters that are different between the two copies of a

website being compared exceeds some pre-specified threshold, currently set to one, i.e. any change.

Select sites below then click the view difference button.

☐ Select all sites	View Difference		
Name	Site	Crawl-Depth	Stay-Within
☐ Alberto O. Mendelzon	☐ http://www.cs.toronto.edu/~mendel/		http://www.cs.toronto.edu/
☐ Alex Borgida	☐ http://www.cs.rutgers.edu/~borgida/		http://www.cs.rutgers.edu/
☒ Alfons Kemper	☒ http://www.db.fni.uni-passau.de/~kemper/		http://www.db.fni.uni-passau.de/
☐ Alon Y. Halevy	☐ http://data.cs.washington.edu/	2	http://data.cs.washington.edu/
	☐ http://www.cs.washington.edu/homes/alon/		http://www.cs.washington.edu/
	☐ http://www.cs.washington.edu/research/semweb/		http://www.cs.washington.edu/
☒ AnHai Motro	☒ http://www.isse.gmu.edu/~ami/		http://www.isse.gmu.edu/
☐ Anastasia Alamaki	☐ http://www-2.cs.cmu.edu/~natassa/		http://www-2.cs.cmu.edu/
☒ AnHai Doan	☒ http://anhai.cs.uiuc.edu/home/		http://anhai.cs.uiuc.edu/
☒ Avigdor Gal	☒ http://few3.technion.ac.il/~avigal/		http://few3.technion.ac.il/
☐ Beng Chin Ooi	☐ http://www.comp.nus.edu.sg/~ooibc/		http://www.comp.nus.edu.sg/

Figure 4.8: Users can further select which sites within both snapshot to perform diff on

This page shows a diff between a snapshot taken on [Thu Mar 25 18:40:30 2004] and a more recent snapshot taken on [Thu Jun 24 18:40:30 2004]

☒ Sort by Name
 ☐ Sort by Title
 ☒ Hide Duplicates
 ☐ Hide Empty Results
 ☐ Hide Course/Class Documents
 ☐ Hide Network

- Researcher: Alfons Kemper Site: [Prof. Alfons Kemper](http://www.db.fni.uni-passau.de/) [Crawl Depth: 1] [Stay Within: <http://www.db.fni.uni-passau.de/>]**
 - No Changes Found
- Researcher: AnHai Doan Site: [AnHai Doan's HomePage](http://anhai.cs.uiuc.edu/home/) [Crawl Depth: 1] [Stay Within: <http://anhai.cs.uiuc.edu/home/>]**
 - [Efficiently Ordering Query Plans for Data Integration](#) added to [Some Recent Talks](#)
 - [Evolving and Self-Managing Data Integration Systems](#) added to [Some Recent Talks](#)
 - [Geometric Foundations for Interval-Based Probabilities](#) added to [Publications by Year](#)
 - [iMAP: Discovering Complex Semantic Matches between Database Schemas](#) added to [AnHai Doan's HomePage](#)
 - [Learning Source Descriptions for Data Integration](#) added to [Some Recent Talks](#)
 - [Learning to Map between Ontologies on the Semantic Web](#) added to [Some Recent Talks](#)
 - [Learning to Map between Structured Representations of Data](#) added to [Some Recent Talks](#)
 - [Learning to Translate between Ontologies](#) added to [Some Recent Talks](#)
 - [Presentation](#) added to [AnHai Doan's HomePage](#)
 - [Reconciling Schemas of Disparate Data Sources: A Machine Learning Approach](#) SIGMOD-2001, by AnHai Doan, added to [Some Recent Talks](#)
 - [Multi-Strategy Learning](#) added to [Some Recent Talks](#)
 - [Report on the Semantic Integration Workshop at the 2nd Int. Semantic Web Conf. \(ISWC-03\)](#) added to [AnHai Doan's HomePage](#)

Figure 4.9: Report on what has changed, papers are shown either as having been recently added or removed

Figure 4.7, figure 4.8 and Figure 4.9 show the process of querying the system for what has changed.

4.4.8 Notification

The system supports on demand, eager and periodic notification [35]. Users can either view reports of changes of the websites being monitored via the web interface whenever they want to, this is on demand notification, or they can elect to receive a report of the sites being monitored via email. A user can further choose to be emailed whenever the monitoring system detects a change in a website the user is interested in, eager notification, or they can elect to be sent a report at some regular interval (e.g. weekly) irrespective of whether there is a change or not, periodic notification.

4.4.9 Scheduler

The scheduler determines how frequently websites are monitored for changes. Currently this is a parameter that is specified by the system administrator. For example in both of our domains all the sites are monitored on a daily basis. The scheduler runs the crawler at the specified intervals, it also passes the results of the crawler to the data-extraction module and caches the extracted data. After each crawling and caching data the scheduler runs the diff module, and then based on the preferences of each user specified within the users xml config file it determines whether to send the user an email containing the differences reported by the diff module. For example, if a user elects to be notified only when there's a change then the scheduler will send the user an email only if the diff module found any changes in any of the websites the user subscribed to monitor. Or if the user elected to be sent reports on a weekly basis, then the scheduler would only send the user an email at the end of each week, if an archiving policy is in effect the user would get a full report on what has changed within the last week otherwise (overwrite policy) the user will only get a report on the most recent changes.

4.5 Monitoring as a Variant of II Systems

Often with monitoring systems, the sites being monitored are within the same domain. It is our belief that since there are usually many sites being monitored, building an information integration system over all these sites would in turn help in building a better monitoring system. In fact we believe that this is the natural way to think of monitoring systems, and below we discuss this further by showing how the challenges faced in building an information integration system are quite similar to those that are faced in building a monitoring system.

In chapter one we listed some of the challenges faced in building an information integration system. We present those challenges again below, but this time we expound on them in the context of building monitoring systems.

- Deciding on a Mediated Schema

As mentioned before, the schema of the global query interface in an information integration system is often referred to as the mediated schema and it is a virtual schema that should capture the salient aspects of the domain. In the context of monitoring system, we can define the mediated scheme as being a virtual schema that should capture the salient aspects of the domain that we're interested in monitoring. In other words deciding on a mediated schema for an information integration system correlates

to deciding on what kind of information to monitor. For example in building a system to monitor information in the finance industry, it is obvious that it would be desirable to have the system monitor changes in stock/bond prices, but what about other information such as dividends, interest rates, news about upcoming mergers, etc. How does one decide what exactly to monitor in this domain, it would be nice to have a very expressive language that would allow the user to specify what kind of information they'd like to monitor but the more expressive the language is the more difficult it becomes to build and support such a system. Ultimately there has to be a limit to what can be monitored, that is it has to be determined before hand what kind of information can be monitored by the system and as with information integration system it is up to experts in the domain to decide which elements of websites will be monitored.

- Source Discovery

The goal of an information integration system is to build a uniform interface over several data sources. The more data sources there are the better the coverage of user queries. The same holds for monitoring systems as well. In certain situations a user might be interested in monitoring just a few sites but often they are not interested as such in just a few particular sites but rather in a particular domain. For example a researcher being interested in new papers published in the field of quantum mechanics. As with information integration systems a focused crawler may be employed or a community site for the domain that lists a lot of the sources. For example in building the monitoring system to monitor the pages of researchers in the database community we were able to obtain the sources from listings of past program committees on the SIGMOD website.

- Wrapper Construction and Maintenance

Programs which are referred to as wrappers are attached to each data source in an information integration system to handle data extraction, formatting and transformations between the data-model of each source and the data-model of the integration system. This correlates to the need for data extraction on the different sites being monitored. For example if we were interested in detecting when companies post their annual reports on their websites, we'd have to have a module in place that can handle extracting the reports irrespective of the how different the layouts of all the websites being monitored are different from each other. This is very similar to the purpose that wrappers serve in information integration systems, thus with monitoring systems as well we need wrappers to handle extraction of the information being monitored for the

different sites.

- Finding Semantic Mappings

The acquisition of semantic mappings is a key obstacle in deploying information integration systems. Semantic mappings specify the relationship between the attributes of the mediated schema and the schema of a local source. The same holds true with monitoring systems, we need to match the type of information we're interested in monitoring with the actual information on the websites. For example in monitoring researcher's websites we might want to detect when their contact information changes in order to detect when they move to another school or company. Some professors could list this information under contact-info, others under address, others under location, thus we'd need to be able to determine these are all semantically the same as the professor's address.

- Join Paths

There are times when a source might not contain all the information needed in the results to be returned to the user in an information integration system. In such an instance it might be necessary to do pose the query over multiple sites simultaneously and then combine the results via a join. This is also true for monitoring system. For example in monitoring websites of researchers, we might also want to monitor other websites of conferences so we can detect when a researcher will be giving talks at conferences and list this under what is new with the researcher. In such an instance we'll be pulling information about researchers from multiple sources and we'd have to figure how to combine all this together via some sort of join.

- Object Matching

Just as with information integration systems, when performing joins the objecting matching problem is encountered. Returning to the previous example, when monitoring websites of conferences such as SIGMOD, how do we match information extracted about a researcher to the website of a researcher being monitored. A researcher's name can be listed differently on different sites, also what if there are multiple researcher's with the same name, how do we tell which particular researcher the extracted information pertains to. Thus with monitoring systems the object matching problem is also encountered.

- Optimization

Query execution and optimization is another important component of information integration systems. Given the numerous data sources that an information integration system might be built over, it is sometimes unreasonable to query all the sources and wait for results to be returned by all the sources before results are returned to the user. Concerns about optimization are even more prevalent with monitoring systems, because not only are there a lot of sources being queried but they're being queried regularly. Thus optimization concerns would not only include how to answer queries like "show me what's new with researcher John Doe in the past one week," but we also have to worry about how often sites should be crawled and cached in order to support queries like this. For example if changes are made to a website on the average of once a month, does it make sense to monitor it daily. Is there anyway to detect how often a site changes, and adjust the monitoring schedule in accordance with this. These are some of the additional optimization issues that are faced in building monitoring systems.

From the above it should be clear that since we encounter the same challenges as in building information integration systems, it would be advantageous to build monitoring systems as an application of information integration systems, i.e. there's no need to reinvent the wheel. Our research group is currently exploring building a monitoring system in this manner on top of our current information integration systems built using MOBS.

4.6 Related Work

There have been several systems developed to monitor web sources for changes. ChangeDetector[10] and WebCQ[35] are two systems with similar functionality to our monitoring system. Both systems allow users to specify a set of urls to be crawled for changes, with users being notified regularly about changes. ChangeDetector leverages machine learning techniques extensively in developing many of it's modules, examples of which include implementing a focused crawler so that only links most likely to contain significant changes are explored during the crawl phase and also for their data extraction modules. The other system WebCQ allows users to not only monitor websites as a whole but particular objects within those sites as well (such as tables, lists, images, words, phrases etc), with each object being cached and monitored individually, also unlike our system and ChangeDetector users are allowed to specify how frequently each source should be monitored. Both of these design choices for WebCQ do not make it scalable both in terms of large number of sites being monitored and having a large number of users. Perhaps the most significant difference between our system

and these other two is that ChangeDetector and WebCQ provide only an overwrite cache policy and thus it is not able to support history queries, such as what has changed between last month and this month. Another system, the At&t Internet difference engine (AIDE) [15] does support history queries, unlike our system caching is done locally on each users machine. Its main feature though is that it provides techniques for performing difference analysis on html documents. Using the markup tags within an html document, the document is broken down into tokens and the tokens are then compared using Hirshbergs solution to the longest common subsequence problem [20], this is the same comparison algorithm tool used in the Unix diff tool [21]. As in Unix diff, the differences found within a web page are highlighted. Unlike the monitoring systems mentioned earlier, AIDE does not incorporate crawling, thus the user must explicitly specify each url they're interested in monitoring even if it is within the same website.

Also noteworthy are the first monitoring tools for websites, which surfaced, in the early 90s as commercial systems, none of which are in existence today. Examples include net, Tierra Highlights and WebWhacker. One of the problems with these early tools was that they treated each web page monitoring request as a unit of notification, thus users that registered a large number of pages with the monitoring service were overwhelmed with frequent email notification messages[35]. An example of a commercial monitoring service that is up and running today is Google's web Alerts[1], which allows users to be notified when the results returned by the google search engine for a particular query changes.

Lastly there has been previous work that does not focus on monitoring systems as a whole but does make contributions that pertain to this paper. We mention two in particular here, the first is a model referred to as continuous adaptive monitoring (CAM) [31]. CAM uses a probabilistic model to determine how often a web page changes and based on this how often a particular source should be monitored for changes as well as how resources should be allocated for this task. The second work discusses efficient matching of user queries for monitoring systems [17]. We mention it here because it is the only other work that we know of that also discusses to some extent treating monitoring systems as an application of information integration system. In particular it mentions coming up with an integrated schema for domains that are being monitored and having each site to be monitored be viewed as an extension of the integrated schema of the domain it belongs to. This is similar to our discussions on finding a mediated schema and semantic mappings for monitoring systems.

In conclusion, we've discussed several other monitoring system and perhaps one of the major advantages our system has over others is its very modular nature and the loose coupling of the modules, which allow for any module to be replaced. We feel that this enables our system to be extended to incorporate the techniques used by these other systems. For

example to have the system do a focused crawl (as was done in ChangeDetector via machine learning) as opposed to it's depth first search only the crawl module of our monitoring system would have to be replaced or modified.

4.7 Summary

In this paper we have presented an overview of our system for site-level web monitoring. We showed how the different tasks such as crawling, caching, data-extraction, and difference modules are integrated together to achieve the goals of the system. We also highlight in our discussions along the way the problems encountered and lessons learnt in solving these problems. Lastly we discussed how monitoring can be viewed as an application of information integration and propose this as possible future work on building a better monitoring system, a direction which our research group is currently exploring. The implementation in perl language of the back end for the monitoring system is included as an appendix.

Appendix A

Perl Code for Monitoring System

```
1  #!/usr/bin/perl
2
3  package MonitorUtils;
4
5  use strict;
6  use HTTP::Request;
7  use HTTP::Response;
8  use LWP::UserAgent;
9  use HTML::TokeParser;
10 use HTTP::Headers;
11
12
13
14 my $ua = LWP::UserAgent->new();
15 $ua->agent("TestBot/0.1");
16
17 #location of sendmail, for sending out emails
18 my $sendmail      = "/usr/sbin/sendmail -t";
19
20 #print flags
21 my $BROWSER = 0;
22 my $DEBUG = 0;
23
24
25 my %crawledLinks = {};
26 my %SiteTitles = {}; #quick hack can do this better, maybe
27 my %changedUrls = {};
28 my %changedSites = {};
29
30
31 #newSave()
32 #details - Takes a snapshot of websites and stores in a time-stamped
folder
33 #arguments - a hash reference, containing information about which sites
should have their snapshot taken
34 # - a string specifying where the time-stamped folder should
be created
35 #returns - string specifying full path to time-stamped snapshot
folder
36 sub newSave
```

```

37 {
38     my $hash_ref = shift @_;
39     my $location = shift @_;
40
41     my $result_filter = "\\.(ps\\.gz|pdf|ps|ppt)\\s*?\\$";
42     my $explore_filter = "\\.(asp|jsp|php).*?\\$|\\.htm\\s*?\\$|\\.html\\s*?\\$|\\.\\w+?\\s*?\\$|\\.\\w+?\\$|\\.\\w+?\\$";
43
44     $hash_ref = getLinksForAllSites($hash_ref, $result_filter,
45                                     $explore_filter);
46     my $dirname = time;
47     $dirname = $location . $dirname . "/";
48     browserPrint("<!-- -->\n");
49     mkdir $dirname, 0777 or die "unable to make dir $dirname, error was
50         : $!";
51     chmod 0777, $dirname or warn "unable to chmod for $dirname, error
52         was: $!";
53
54     my $k;
55     my %sites = %{$hash_ref};
56     foreach $k (keys %sites)
57     {
58         my $file_name = $k;
59         $file_name =~ s#/#\\#g;
60         #$file_name = $dirname . "_" . $file_name;
61
62         browserPrint("<!-- -->\n");
63         writeSiteFile($dirname . "_" . $file_name, $k, $hash_ref);
64         writeUrlFile($dirname . "okay" . "_" . $file_name, $k,
65                     $hash_ref, "okay");
66         writeUrlFile($dirname . "broken" . "_" . $file_name, $k,
67                     $hash_ref, "broken");
68     }
69     return $dirname;
70 }
71
72 #getDirList()
73 #details - Returns the contents of all files/folders in a directory
74 #arguments - directory to b read
75 #returns - string specifying full path to time-stamped snapshot
76           folder
77 sub getDirList
78 {
79     my $the_dir = shift @_;
80     opendir DIRHANDLE, $the_dir or die "unable to open directory
81         $the_dir, error was: $!";
82     my @allfiles = grep !/^\\.\\.$?$/, readdir DIRHANDLE;
83     closedir DIRHANDLE;
84     return (sort {$b <=> $a} @allfiles);
85 }
86
87 #diffSavedSites()
88 #details - Returns the difference between list of links cached for
89           each site
90 #arguments - 2 hash_references (keys = site-urls, values = array of
91           links found extracted from site)

```

```

84  #returns - a hash_reference containing the difference btw the links
      for each site
85  sub diffSavedSites
86  {
87      my $hash_ref1 = shift @_;
88      my $hash_ref2 = shift @_;
89      my %sites1 = %{$hash_ref1};
90      my %sites2 = %{$hash_ref2};
91
92      my $k;
93      foreach $k (keys %sites1) {
94          unless (exists($sites2{$k}))
95          {
96              debugPrint("Warning: doing diff, but no match for $k");
97              next;
98          }
99          my %temp1 = %{$sites1{$k}};
100         my %temp2 = %{$sites2{$k}};
101
102         my @init_links = diffLinks($temp1{"links"}, $temp2{"links"});
103         my @broken = diffLinks($temp1{"okay"}, $temp2{"okay"});
104         my @links;
105         my %brokenHash;
106         my $b;
107         foreach $b (@broken) {
108             my @info = @{$b};
109             $brokenHash{$info[0]} = "";
110         }
111         foreach $b (@init_links) {
112             my @info = @{$b};
113             next if (exists($brokenHash{$info[2]}));
114             push @links, $b;
115         }
116         $temp1{"links"} = \@links;
117         $temp1{"broken"} = \@broken;
118
119         $sites1{$k} = \%temp1;
120     }
121     $hash_ref1 = \%sites1;
122     return $hash_ref1;
123 }
124
125 #readSavedSites()
126 #details - Reads in a saved snapshot into a hash-reference
127 #arguments - hash-reference containing list of sites to read in
128 #returns - hash-reference with snapshot data associated with each
      site
129 sub readSavedSites
130 {
131     my $hash_ref = shift @_;
132     my $location = shift @_;
133     my %sites = %{$hash_ref};
134
135     my $k;
136     foreach $k (keys %sites)
137     {
138         my $file_name = $k;

```

```

139     $file_name =~ s#/#\\#g;
140     $$file_name = $location . "_" . $file_name ;
141     $hash_ref = readSiteFile($location . "_" . $file_name, $k,
        $hash_ref);
142     $hash_ref = readUrlFile($location . "okay" . "_" . $file_name,
        $k, $hash_ref, "okay");
143     $hash_ref = readUrlFile($location . "broken" . "_" . $file_name
        , $k, $hash_ref, "broken");
144 }
145 return $hash_ref;
146 }
147
148 #getLinks()
149 #details - Extracts all the links in a page
150 #arguments - string containing url of page
151 #returns - a list of links found on the page
152 sub getLinks
153 {
154     my $page_url = shift @_;
155     browserPrint("<!-- -->\n");
156     my $req;
157     my $res;
158     my $i = 0;
159     while ($i < 2) { #Attempts to get a page twice.
160         browserPrint("<!-- -->\n");
161         $req = HTTP::Request->new('GET' => $page_url);
162         $res = $ua->request($req);
163         if($res->is_success) {
164             last;
165         }
166         $i++;
167     }
168     unless($res->is_success) { # If page is not present flag an error.
169         debugPrint("ERROR: Unable to retrieve source page: $page_url\n"
            );
170         return (-1, -1);
171     }
172     my $title = $res->title;
173     if (defined($SiteTitles{$page_url})) {
174         $SiteTitles{$page_url} = $title;
175     }
176     my $p = HTML::TokeParser->new($res->content_ref);
177
178     my @link_info = ();
179     while (my $token = $p->get_tag("a")) {
180         my $t;
181         my $url = $token->[1]{href};
182         my $text = $p->get_trimmed_text("/a");
183         my $p_url = URI->new($page_url);
184         my $ref_uri = URI->new($url);
185         my $abs_url = $ref_uri->abs($p_url);
186         browserPrint("<!-- -->\n");
187         # Each node contains the absolute url,
            the anchor text, current url and
            and title of current url.
188         push @link_info, [$abs_url->canonical, $text, $p_url->canonical
            , $title];

```

```

189     }
190     return @link_info;
191 }
192
193 #filterLinks()
194 #details - Returns all links matching a particular filter (regular
           expression)
195 #arguments - A list of links
196 #          - filter specified by regular expression
197 #returns  - A list of links
198 sub filterLinks
199 {
200     my $filter = shift @_;
201     my @link_info = @_;
202     return @link_info unless (defined($filter) && $filter =~ /\S/);
203     my $l;
204     my @filtered_link_info = ();
205     foreach $l (@link_info) {
206         my @info = @{$l};
207         my $url = $info[0];
208         browserPrint("<!-- -->\n");
209         #quickDebug($url, "Found in filterLinks before Filtering");
210         next if ($url !~ /$filter/i);
211         #quickDebug($url, "Found in filterLinks after Filtering");
212         push @filtered_link_info, $l;
213     }
214     return @filtered_link_info;
215 }
216
217 #filterOutLinks()
218 #details - Returns all the links that do not match specified filter
219 #arguments - A list of links
220 #          - filter specified by regular expression
221 #          - list of integers, specifying which part of link-
           datastructure to filter on
222 #returns  - a list of links
223 sub filterOutLinks
224 {
225     my @link_info = @shift @_;
226     my $filter = shift @_;
227     my @keys = @_;
228
229     return @link_info unless (defined($filter) && $filter =~ /\S/);
230     my $l;
231     my @filtered_link_info = ();
232     foreach $l (@link_info) {
233         my @info = @{$l};
234         my $found = 0;
235         my $k;
236         foreach $k (@keys) {
237             my $url = $info[$k];
238             if ($url =~ /$filter/i)
239             {
240                 $found = 1;
241                 last;
242             }
243         }

```

```

244         browserPrint("<!-- -->\n");
245         #quickDebug($url, "Found in filterLinks before Filtering");
246         next if ($found == 1);
247         #quickDebug($url, "Found in filterLinks after Filtering");
248         push @filtered_link_info, $l;
249     }
250     return @filtered_link_info;
251 }
252
253 #removeDuplicats()
254 #details - Removes all duplicates found in list of links
255 #arguments - A list of links
256 #returns - A list of links
257 sub removeDuplicateLinks
258 {
259     my @link_info = @_;
260     my @link;
261     my %linkTable;
262
263     my $i = 0;
264     my $l;
265     foreach $l (@link_info) {
266         my @info = @{$l};
267         my $url = $info[0];
268         next if ($linkTable{lc($url)} == 1);
269         $linkTable{lc($url)} = 1;
270         push @link, $l;
271
272         #if ($linkTable{$url} == 1){
273             #debugPrint("Found duplicate: $url\n");
274             #splice @link_info, $i, 1;
275             #next;
276         #}
277         $linkTable{$url} = 1;
278         $i++;
279     }
280     return @link;
281 }
282
283 sub filterHash
284 {
285     my %sourceHash = %{shift @_};
286     my @filterKeys = @_;
287     my %targetHash;
288
289     my $k;
290     foreach $k (@filterKeys) {
291         if (exists($sourceHash{$k})) {
292             $targetHash{$k} = $sourceHash{$k};
293         }
294     }
295     return \%targetHash;
296 }
297 #For a particular website crawl for the given depth and retrieve all
    the relevant urls.
298 sub getLinksForSite
299 {

```

```

300     my ($link_info_ref, $depth, $resultsFilter, @crawlFilter) = @_;
301     my @link_info = @{$link_info_ref};
302     my $site = $link_info[0];
303     $crawledLinks{$site} = 1;
304     browserPrint("<!-- -->\n");
305     my @links = getLinks($site); # get all the links in the site.
306     my @okayUrls;
307     my @brokenUrls;
308     if (defined($links[0]) && $links[0] == -1) { # if there are no
        links the url is broken
309         push @brokenUrls, $link_info_ref;
310         @links = ();
311     }
312     else { # else it is fine
313         push @okayUrls, $link_info_ref;
314     }
315
316     my @results = filterLinks($resultsFilter, @links);
317     return (\@results, \@okayUrls, \@brokenUrls) if ($depth == 0);
318
319     foreach my $c (@crawlFilter) {
320         @links = filterLinks($c, @links); #which links to crawl, e.g.
        only *htm // *html
321     }
322
323     foreach my $l (@links) {
324         my @info = @{$l}; # Each info element has an array of elements
        representing the tag.
325         my $url = $info[0];
326         my $url2 = $url . "/";
327         browserPrint("<!-- -->\n");
328         next if (($crawledLinks{$url} == 1 || $crawledLinks{$url2} ==
            1) && debugPrint("Skipping: $url\n"));
329         my ($res_ref, $ok_ref, $brk_ref) = getLinksForSite($l, $depth
            -1, $resultsFilter, @crawlFilter);
330         my @temp = @{$res_ref};
331         my @temp2 = @{$ok_ref};
332         my @temp3 = @{$brk_ref};
333
334         if (defined($temp[0])){
335             push @results, @temp;
336         }
337         if (defined($temp2[0])){
338             push @okayUrls, @temp2;
339         }
340         if (defined($temp3[0])){
341             push @brokenUrls, @temp3;
342         }
343     }
344     return (\@results, \@okayUrls, \@brokenUrls);
345 }
346
347 # Returns a hash of sites and relevant links in it. Each element in the
    hash is referenced by the
348 # site. Then the array of links is referenced by the links tag.
349 sub getLinksForAllSites
350 {

```

```

351     my $hash_ref = shift @_;
352     my $resultFilter = shift @_;
353     my $exploreFilter = shift @_;
354     my %sites = %{$hash_ref};
355     %crawledLinks = {};
356     my $i = 1;
357     my $numberOfSites = scalar(keys %sites);
358     foreach my $k (keys %sites){
359         my %temp = %{$sites{$k}};
360         my $depth = $temp{"depth"};
361         my $within = $temp{"allow"};
362         unless ( defined($depth) && ($depth =~ /\d+$/) ) {
363             $depth = 1000; #large enough to consider infinite depth
364         }
365         unless (defined($within)) {
366             $within = "";
367         }
368         # Why [$k, "", "", ""].
369         my ($res_ref, $ok_ref, $brk_ref) = getLinksForSite([$k, "", "",
370             ""], $depth, $resultFilter, $exploreFilter, $within);
371         $temp{"links"} = $res_ref;
372         $temp{"okay"} = $ok_ref;
373         $temp{"broken"} = $brk_ref;
374         $sites{$k} = \%temp;
375         %crawledLinks = {};
376         debugPrint("*****Done with site$i: $k\n
377             *****");
378         browserPrint("Done with site: $k, completed $i out of
379             $numberOfSites<br>\n");
380         $i++;
381     }
382     return \%sites;
383 }
384
385 sub getKeysSortedByName
386 {
387     my %hash = %{$shift @_};
388     my @sortedKeys = keys %hash;
389     @sortedKeys = sort { lc(${$hash{$a}}{"name"}) cmp lc(${$hash{$b}}{"name"}) } @sortedKeys;
390     return @sortedKeys;
391 }
392
393 sub printSnapshotHtml
394 {
395     my $hash_ref = shift @_;
396     my $type = shift @_;
397     my $hideEmpty = shift @_;
398     my $hide = shift @_;
399     my $sort = shift @_;
400     my $course = shift @_;
401     my $network = shift @_;
402     my %sites = %{$hash_ref};
403
404     my $site_name;
405     print "<ol>\n";

```

```

404 my @sortedSites = keys %sites;
405 if ($sort eq "name") {
406     @sortedSites = getKeysSortedByName($hash_ref);
407 }
408 else
409 {
410     @sortedSites = sort { lc(${sites{$a}}{"title"}) cmp lc(${
411         sites{$b}}{"title"}) } @sortedSites;
412 }
413 foreach $site_name(@sortedSites) {
414     my %temp = %{(sites{$site_name})};
415     my @links = @{($temp{"links"})};
416     #my @okay_links = @{($temp{"okay"})};
417     my @broken_links = @{($temp{"broken"})};
418     my $d = $temp{"depth"};
419     my $w = $temp{"allow"};
420     my $tit = $temp{"title"};
421     my $name = $temp{"name"};
422
423     if ($course == 1){
424         @links = filterOutLinks(\@links, "homework|exam|midterm|
425             course|handout|fall|spring|winter|lecture", 1, 3);
426     }
427     next if ((scalar(@links) == 0) && ($hideEmpty == 1));
428
429     print <<EOF2;
430
431         <p><li><b>Researcher: $name Site: <a href="
432             $site_name">$tit</a> [Crawl Depth: $d] [Stay
433             Within: $w]</b>
434
435 EOF2
436
437     print "\n<ul>";
438     my $l;
439     my $td;
440     if ($hide == 1) {
441         @links = removeDuplicateLinks(@links);
442     }
443
444     my @sortedLinks = sort { lc(${a}[1]) cmp lc(${b}[1]) } @links;
445     if (scalar(@sortedLinks) == 0) {
446         if ($type eq "diff") {
447             print "<li><b>No Changes Found\n</b>";
448         }
449         else
450         {
451             print "<li><b>No pdf/ps links found\n</b>";
452         }
453     }
454
455     foreach $l (@sortedLinks) {
456         my @info = @{$l};
457         my $u = $info[0];
458         my $t = $info[1];
459         my $p = $info[2];
460         my $ti = $info[3];
461         my $edit = " found on ";

```

```

457         if ($type eq "diff") {
458             $edit = " " . $info[4] . " ";
459         }
460         print <<EOF3;
461             <li><a href="$u">$t</a> $edit <a href="$p">$ti
462                 </a>
463     EOF3
464     }
465     if ($network == 1) {
466
467         @sortedLinks = sort { lc(${a}[1]) cmp lc(${b}[1]) }
468             @broken_links;
469         if (scalar(@sortedLinks) > 0) {
470             if ($type eq "diff") {
471                 print "<li><b>Below is a list of links that were not
472                     crawled in BOTH snapshots (due to network errors e.g
473                     . delays or broken links), these links were not
474                     considered for differences: \n";
475                 print "[added] implies the link was present in the more
476                     recent snapshot but not in the former (vice versa
477                     for [removed]):</b><ul>\n";
478             }
479             else
480             {
481                 print "<li><b>Below is a list of links not crawled due
482                     to network errors (e.g. delays or broken links)</b><
483                     ul>:\n";
484             }
485
486             foreach $l (@sortedLinks) {
487                 my @info = @{$l};
488                 my $u = $info[0];
489                 my $t = $info[1];
490                 my $p = $info[2];
491                 my $ti = $info[3];
492                 my $edit = " found on ";
493                 if ($type eq "diff") {
494                     $edit = " " . $info[4] . " ";
495                 }
496                 print <<EOF4;
497                     <li><a href="$u">$t</a> $edit <a href="$p">$ti
498                         </a>
499     EOF4
500             }
501             print "</ul>\n";
502         }
503         print "</ul></p>\n";
504     }
505     print "</ol>\n";
506 }
507
508 sub getSnapshotEmailHtml
509 {
510     my $hash_ref = shift @_;

```

```

504 my $type = shift @_ ;
505 my $hideEmpty = shift @_ ;
506 my $hide = shift @_ ;
507 my $sort = shift @_ ;
508 my @rules = @_ ;
509 my %sites = %{$hash_ref};
510 my %results_hash;
511 my $diffResult;
512
513 my $site_name;
514 $diffResult = "<ol>\n";
515
516 my @sortedSites = keys %sites;
517 if ($sort eq "name") {
518     @sortedSites = getKeysSortedByName($hash_ref);
519 }
520 else
521 {
522     @sortedSites = sort { lc(${$sites}{$a}){"title"}) cmp lc(${$sites}{$b}){"title"}) } @sortedSites;
523 }
524
525 foreach $site_name(@sortedSites) {
526     my %temp = %{$sites{$site_name}};
527     my @links = @{$temp{"links"}};
528     my $d = $temp{"depth"};
529     my $w = $temp{"allow"};
530     my $tit = $temp{"title"};
531     my $name = $temp{"name"};
532
533     if (defined($rules[0])){
534         my $r;
535         foreach $r (@rules) {
536             #rules table
537             if ($r eq "rule1") {
538                 @links = filterOutLinks(\@links, "homework|exam|
                    midterm|course|handout|fall|spring|winter", 1,
                    3);
539             }
540         }
541     }
542
543     next if ((scalar(@links) == 0) && ($hideEmpty == 1));
544
545     $diffResult .= "<p><li><b>Researcher: $name Site: <a href='
                    $site_name'>$tit</a> [Crawl Depth: $d] [Stay Within: $w]</b>
                    ";
546
547     $diffResult .= "\n<ul>";
548     my $l;
549     my $td;
550     if ($hide == 1) {
551         @links = removeDuplicateLinks(@links);
552     }
553 }
554

```

```

555         my @sortedLinks = sort { lc(${$a}[1]) cmp lc(${$b}[1]) } @links
556         ;
557         if (scalar(@sortedLinks) == 0) {
558             $diffResult .= "<li><b>No Changes Found\n</b>";
559         }
560         foreach $l (@sortedLinks) {
561             my @info = @{$l};
562             my $u = $info[0];
563             my $t = $info[1];
564             my $p = $info[2];
565             my $ti = $info[3];
566             my $edit = " found on ";
567             if ($type eq "diff") {
568                 $edit = " " . $info[4] . " ";
569             }
570             $diffResult .= "<li><a href='$u'>$t</a> $edit <a href='$p'>
571                 $ti</a>\n";
572         }
573         $diffResult .= "</ul></p>\n";
574     }
575     return $diffResult;
576 }
577 sub getSnapshotEmailText
578 {
579     my $hash_ref = shift @_;
580     my $type = shift @_;
581     my $hideEmpty = shift @_;
582     my $hide = shift @_;
583     my $sort = shift @_;
584     my @rules = @_;
585     my %sites = %{$hash_ref};
586     my %results_hash;
587     my $diffResult;
588
589     my $site_name;
590     $diffResult = "";
591
592     my @sortedSites = keys %sites;
593     if ($sort eq "name") {
594         @sortedSites = getKeysSortedByName($hash_ref);
595     }
596     else
597     {
598         @sortedSites = sort { lc(${$sites{$a}}{"title"}) cmp lc(${$
599             $sites{$b}}{"title"}) } @sortedSites;
600     }
601
602     foreach $site_name(@sortedSites) {
603         my %temp = %{$sites{$site_name}};
604         my @links = @{$temp{"links"}};
605         my $d = $temp{"depth"};
606         my $w = $temp{"allow"};
607         my $tit = $temp{"title"};
608         my $name = $temp{"name"};

```

```

609         if (defined($rules[0])){
610             my $r;
611             foreach $r (@rules) {
612                 #rules table
613                 if ($r eq "rule1") {
614                     @links = filterOutLinks(\@links, "homework|exam|
                        midterm|course|handout|fall|spring|winter", 1,
                        3);
615                 }
616             }
617         }
618     }
619     next if ((scalar(@links) == 0) && ($hideEmpty == 1));
620
621     $diffResult .= "Researcher: $name SiteName: $tit SiteUrl:
        $site_name [Crawl Depth: $d] [Stay Within: $w]\n";
622
623     my $l;
624     my $td;
625     if ($hide == 1) {
626         @links = removeDuplicateLinks(@links);
627     }
628
629     my @sortedLinks = sort { lc(${$a}[1]) cmp lc(${$b}[1]) } @links
        ;
630     if (scalar(@sortedLinks) == 0) {
631         $diffResult .= "    No Changes Found\n";
632     }
633     foreach $l (@sortedLinks) {
634         my @info = @{$l};
635         my $u = $info[0];
636         my $t = $info[1];
637         my $p = $info[2];
638         my $ti = $info[3];
639         my $edit = " found on ";
640         if ($type eq "diff") {
641             $edit = " " . $info[4] . " ";
642         }
643         $diffResult .= "    PaperName: $t PaperUrl: $u $edit
            PageName:$ti PageUrl: $p\n";
644     }
645     $diffResult .= "\n";
646 }
647 $diffResult .= "\n";
648 return $diffResult;
649 }
650
651 sub checkUrlFormat
652 {
653     my $url = shift @_;
654     if ($url =~ /^https?:\/\/\/\S+?\.\S+?$/ ) {
655         return 1;
656     }
657     else
658     {
659         warn("Warning: illegal url format for $url\n");
660         return 0;

```

```

661     }
662 }
663
664 sub readSiteFile
665 {
666     my $file_name = shift @_;
667     my $site_name = shift @_;
668     my $hash_ref = shift @_;
669     my %sites = %{$hash_ref};
670     my $k;
671     my $name = $sites{$site_name}->{"name"};
672
673     checkUrlFormat($site_name);
674
675     browserPrint("<!-- -->\n");
676     open(READ, "<$file_name") or die "unable to open for reading site
        info: $file_name";
677
678     my $line = <READ>;
679     $line =~ s/\n//;
680     my ($url, $depth, $title, @within) = split /\|/, $line;
681     my $with = join '|', @within;
682     die "file contains link for $url and not for $site_name" if (
        $site_name ne $url);
683     my %temp;
684     $temp{"depth"} = $depth;
685     $temp{"allow"} = $with;
686     $temp{"title"} = $title;
687     if (defined($name)) {
688         $temp{"name"} = $name;
689     }
690     my @links;
691     while(<READ>) {
692         $line = $_;
693         $line =~ s/\n//;
694         next if ($line =~ /\^s*$/);
695         my ($u, $text, $page_url, $page_title) = split /\|/, $line;
696         checkUrlFormat($page_url);
697         push @links, [$u, $text, $page_url, $page_title];
698         browserPrint("<!-- -->\n");
699     }
700     $temp{"links"} = \@links;
701     $sites{$site_name} = \%temp;
702     close READ;
703     return \%sites;
704 }
705
706 sub writeSiteFile
707 {
708     my $file_name = shift @_;
709     my $site_name = shift @_;
710     my $hash_ref = shift @_;
711     my %sites = %{$hash_ref};
712     browserPrint("<!-- -->\n");
713
714     checkUrlFormat($site_name);
715

```

```

716     open(WRITE, ">$file_name") or die "unable to open for writing site
       info: $file_name";
717
718     my %temp = %{$sites{$site_name}};
719     my @links = @{$temp{"links"}};
720     my $d = $temp{"depth"};
721     my $w = $temp{"allow"};
722     my $title = $site_name;
723     if ( defined($SiteTitles{$site_name}) && ($SiteTitles{$site_name}
       !~ /\s*$/) )
724     {
725         $title = $SiteTitles{$site_name};
726     }
727
728     print WRITE "$site_name|$d|$title|$w\n";
729     my $l;
730     foreach $l (@links) {
731         browserPrint("<!-- -->\n");
732         my @info = @{$l};
733         $title = $info[2];
734         my $text = $info[0];
735         if ( defined($info[1]) && ($info[1] !~ /\s*$/) )
736         {
737             $text = $info[1];
738         }
739         if ( defined($info[3]) && ($info[3] !~ /\s*$/) )
740         {
741             $title = $info[3];
742         }
743         print WRITE "$info[0]|$text|$info[2]|$title\n";
744     }
745     close WRITE;
746     chmod 0664, $file_name or warn "unable to chmod for $file_name,
       error was: $!";
747 }
748
749 sub readUrlFile
750 {
751     my $file_name = shift @_;
752     my $site_name = shift @_;
753     my $hash_ref = shift @_;
754     my $type = shift @_;
755     my %sites = %{$hash_ref};
756     my $k;
757     my $name = $sites{$site_name}->{"name"};
758
759     checkUrlFormat($site_name);
760
761     browserPrint("<!-- -->\n");
762     open(READ, "<$file_name") or die "unable to open for reading site
       info: $file_name";
763
764     my %temp = %{$sites{$site_name}};
765     my @links;
766     my $line;
767     while(<READ>) {
768         $line = $_;

```

```

769         $line =~ s/\n//;
770         next if ($line =~ /\s*$/);
771         my ($u, $text, $page_url, $page_title) = split /\|/, $line;
772         checkUrlFormat($page_url);
773         push @links, [$u, $text, $page_url, $page_title];
774         browserPrint("<!-- -->\n");
775     }
776     $temp{$type} = \@links;
777     $sites{$site_name} = \%temp;
778     $$SiteTitles{$site_name} = "";
779     close READ;
780     return \%sites;
781 }
782
783 sub writeUrlFile
784 {
785     my $file_name = shift @_;
786     my $site_name = shift @_;
787     my $hash_ref = shift @_;
788     my $type = shift @_;
789     my %sites = %{$hash_ref};
790     browserPrint("<!-- -->\n");
791
792     checkUrlFormat($site_name);
793
794     open(WRITE, ">$file_name") or die "unable to open for writing url
795         info: $file_name";
796
797     my %temp = %{$sites{$site_name}};
798     my @links = @{$temp{$type}};
799     my $title = $site_name;
800     if ( defined($SiteTitles{$site_name}) && ($SiteTitles{$site_name}
801         !~ /\s*$/))
802     {
803         $title = $SiteTitles{$site_name};
804     }
805
806     my $l;
807     foreach $l (@links) {
808         browserPrint("<!-- -->\n");
809         my @info = @{$l};
810         $title = $info[2];
811         my $text = $info[0];
812         if ( defined($info[1]) && ($info[1] !~ /\s*$/))
813         {
814             $text = $info[1];
815         }
816         if ( defined($info[3]) && ($info[3] !~ /\s*$/))
817         {
818             $title = $info[3];
819         }
820         print WRITE "$info[0]|$text|$info[2]|$title\n";
821     }
822     close WRITE;
823     chmod 0664, $file_name or warn "unable to chmod for $file_name,
824         error was: $!";
825 }

```

```

823
824 sub sortLinks
825 {
826     my @link_info = @_;
827     my @sorted = sort { ($$a[0] . $$a[2]) cmp ($$b[0] . $$b[2]) }
        @link_info;
828     return @sorted;
829 }
830
831 sub diffLinks
832 {
833     my ($refA, $refB) = @_;
834     my @listA = sortLinks(@{$refA});
835     my @listB = sortLinks(@{$refB});
836
837     my $i = 0; my $j = 0;
838     my @diffResult;
839     while($i < scalar(@listA) || $j < scalar(@listB))
840     {
841         if ($i >= scalar(@listA)) {
842             my @info = @{$listB[$j]};
843             push @info, "added to";
844             push @diffResult, \@info;
845             $j++;
846             next;
847         }
848
849         if ($j >= scalar(@listB)) {
850             my @info = @{$listA[$i]};
851             push @info, "removed from";
852             push @diffResult, \@info;
853             $i++;
854             next;
855         }
856
857         my @infoA = @{$listA[$i]};
858         my @infoB = @{$listB[$j]};
859         my $urlA = $infoA[0];
860         my $urlB = $infoB[0];
861         if (($urlA cmp $urlB) == 0)
862         {
863             $j++; $i++;
864         }
865         elsif (($urlA cmp $urlB) < 0) {
866             push @infoA, "removed from";
867             push @diffResult, \@infoA;
868             $i++;
869         }
870         else
871         {
872             push @infoB, "added to";
873             push @diffResult, \@infoB;
874             $j++;
875         }
876     }
877     return @diffResult;
878 }

```

```

879
880 sub printLinkInfo
881 {
882     my @link_info = @_;
883     my $l;
884     foreach $l (@link_info) {
885         my @info = @{$l};
886         my $i;
887         #debugPrint("$info[1]");
888         for ($i = 0; $i < scalar(@info); $i++) {
889             debugPrint("$i:\t$info[$i]\t");
890         }
891         debugPrint("\n");
892     }
893 }
894
895 sub translateUrlToFileName
896 {
897     my $file_name = shift @_;
898     $file_name =~ s#/#\\#g;
899     $file_name = "_" . $file_name;
900     return $file_name;
901 }
902
903 sub readCachedPage
904 {
905     my $file_name = shift @_;
906     open (INIT, "<$file_name") or die "unabale to open cached page
907         $file_name for reading, error was: $!";
908     my $content = (join "", <INIT>);
909     return \$content;
910 }
911
912 sub writeCachedPage
913 {
914     my ($file_name, $contents_ref) = @_;
915     my $contents = $$contents_ref;
916
917     die "unable to write undefined value $contents" unless defined(
918         $contents);
919
920     open (INIT, ">$file_name") or die "couldn't open init file
921         $file_name for writing, error was: $!";
922
923     print INIT "$contents";
924     close INIT;
925     chmod 0664, $file_name or warn "unable to chmod for $file_name,
926         error was: $!";
927     #chmod 0666, $file_name or die "unable to chmod for $file_name,
928         error was: $!";
929 }
930
931 sub sendMail
932 {
933     my ($to, $from, $subject, $text_message, $html_message) = @_;
934
935     my $email = <<THE_EMAIL;

```

```

931 MIME-Version: 1.0
932 Content-Type: multipart/alternative; boundary="
    _jkkdsffds32432dlkjifewks_"
933 To: $to
934 From: $from
935 Subject: $subject
936
937 --_jkkdsffds32432dlkjifewks_
938 Content-Type: text/plain; charset="iso-8859-1"
939
940 $text_message
941
942 --_jkkdsffds32432dlkjifewks_
943 Content-Type: text/html; charset="iso-8859-1"
944
945 <html>
946 <body>
947 <!-------
948 We apologize if you see this message. The
949 email was intended to display either plain
950 text or HTML, not both. You may ignore the
951 rest of the email. The plain text above is
952 the complete message intended to reach you.
953 ----->
954
955 $html_message
956 </body>
957 </html>
958
959 --_jkkdsffds32432dlkjifewks_--
960 THE_EMAIL
961
962     open(MAIL, "|$sendmail") or die "couldn't open sendmail, error was:
        $!";
963     print MAIL $email;
964     close(MAIL);
965 }
966
967 sub parseXml
968 {
969     my $content = shift @_;
970     my $hash_ref = parseXmlHelper($content, {});
971     my %hash = %{$hash_ref};
972
973     die "<xml-document> expected\n" unless exists($hash_ref->{"xml-
        document"});
974     $hash_ref = ${$hash_ref->{"xml-document"}}[0];
975     return $hash_ref;
976 }
977
978 sub parseXmlHelper
979 {
980     my $content = shift @_;
981     my $hash_ref = shift @_;
982     my %xml_hash = %{$hash_ref};
983
984     if (!defined($content) || ($content =~ /\s*$/s)){

```

```

985         return $hash_ref;
986     }
987
988     if ($content !~ /\s*<\s*(\S?)\s*>\s*(.*?)\s*<(\s*\s*)\s*>\s*/
989         s) {
990         die "error while parsing xml file\n";
991     }
992
993     my $tag_name = $1;
994     my $tag_content = $2;
995     $content = $';
996
997     #debugPrint("TagName: #tag_name#\n");
998     #debugPrint("TagContent: #tag_content#\n");
999     #debugPrint("Content: #content#\n");
1000
1001     if (!exists($xml_hash{$tag_name}))
1002     {
1003         $xml_hash{$tag_name} = [];
1004     }
1005
1006     #nested tags
1007     if ($tag_content =~ /\s*</s) {
1008         $tag_content = parseXmlHelper($tag_content, {});
1009     }
1010
1011     my @tag_array = @{$xml_hash{$tag_name}};
1012     push @tag_array, $tag_content;
1013     $xml_hash{$tag_name} = \@tag_array;
1014
1015     $hash_ref = parseXmlHelper($content, \%xml_hash);
1016     return $hash_ref;
1017 }
1018
1019 sub generateXml
1020 {
1021     my $hash_ref = shift @_;
1022     my $content = generateXmlHelper($hash_ref, 1);
1023     $content = "<xml-document>\n" . $content . "</xml-document>\n";
1024     return $content;
1025 }
1026
1027 sub generateXmlHelper
1028 {
1029     my $hash_ref = shift @_;
1030     my $tab_level = shift @_;
1031     die "generateXmlHelper: expected hash reference\n" unless (ref(
1032         $hash_ref) eq "HASH");
1033     my %xml_hash = %{$hash_ref};
1034     my $content = "";
1035
1036     my $x;
1037     foreach $x (keys %xml_hash) {
1038         die "generateXmlHelper: expected array reference\n" unless (ref

```

```

1039         my @tag_array = @{$xml_hash{$x}};
1040
1041         my $t;
1042         foreach $t (@tag_array) {
1043             for (my $i = 0; $i < $tab_level; $i++) {
1044                 $content .= "\t";
1045             }
1046             $content .= "<$x>";
1047             if (ref($t) eq "HASH"){ #nested tag
1048                 $content = $content . "\n";
1049                 $content = $content. generateXmlHelper($t, $tab_level
1050                     +1);
1051                 for (my $i = 0; $i < $tab_level; $i++) {
1052                     $content .= "\t";
1053                 }
1054                 $content .= "</$x>\n";
1055             }
1056             else {
1057                 $content .= $t;
1058                 $content .= "</$x>\n"
1059             }
1060         }
1061
1062         return $content;
1063     }
1064
1065     sub convertXmlHashToUserHash
1066     {
1067         my $hash_ref = shift @_;
1068         my %xml_hash = %{$hash_ref};
1069         my %user_hash;
1070
1071         die "expected <user>\n" unless exists($xml_hash{"user"});
1072         my @tag_array = @{$xml_hash{"user"}};
1073
1074         my $t;
1075         foreach $t (@tag_array) {
1076             my %temp;
1077             $temp{"name"} = ${$t->{"name"}}[0];
1078             $temp{"notify"} = ${$t->{"notify"}}[0];
1079             $temp{"url"} = $t->{"url"};
1080             $user_hash{${$t->{"email"}}[0]} = \%temp;
1081         }
1082         return \%user_hash;
1083     }
1084
1085     sub convertUserHashToXmlHash
1086     {
1087         my $hash_ref = shift @_;
1088         my %user_hash = %{$hash_ref};
1089         my %xml_hash;
1090
1091         my $x;
1092         my @userArray;
1093         foreach $x (keys %user_hash) {
1094             my %temp = %{$user_hash{$x}};

```

```

1095         my %info_hash;
1096
1097         $info_hash{"email"} = [$x];
1098         $info_hash{"name"} = [$temp{"name"}];
1099         $info_hash{"url"} = $temp{"url"};
1100         $info_hash{"notify"} = [$temp{"notify"}];
1101
1102         push @userArray, \%info_hash;
1103     }
1104     $xml_hash{"user"} = \@userArray;
1105     return \%xml_hash;
1106 }
1107
1108 sub convertXmlHashToProfessorHash
1109 {
1110     my $hash_ref = shift @_;
1111     my %xml_hash = %{$hash_ref};
1112     my %professor_hash;
1113
1114     die "expected <professor>\n" unless exists($xml_hash{"professor"});
1115     my @tag_array = @{$xml_hash{"professor"}};
1116
1117     my $t;
1118     foreach $t (@tag_array){
1119         my %temp;
1120         $temp{"name"} = ${$t->"name"}[0];
1121
1122         die "expected <site>\n" unless exists($t->"site");
1123         my @tag_array2 = @{$t->"site"};
1124         my %result_hash;
1125
1126         my $t2;
1127         my %temp2;
1128         foreach $t2 (@tag_array2) {
1129             my %temp3;
1130             if (defined(${ $t2->"depth" }[0])) {
1131                 $temp3{"depth"} = ${ $t2->"depth" }[0];
1132             }
1133             if (defined(${ $t2->"allow" }[0])) {
1134                 $temp3{"allow"} = ${ $t2->"allow" }[0];
1135             }
1136             $temp2{ ${ $t2->"url" }[0] } = \%temp3;
1137         }
1138         $temp{"sites"} = \%temp2;
1139         $professor_hash{ ${ $t->"email" }[0] } = \%temp;
1140     }
1141     return \%professor_hash;
1142 }
1143
1144 sub convertProfessorHashToXmlHash
1145 {
1146     my $hash_ref = shift @_;
1147     my %professor_hash = %{$hash_ref};
1148     my %xml_hash;
1149
1150     my $k;
1151     my @professor_array;

```

```

1152     foreach $k (keys %professor_hash) {
1153         my %temp = %{$professor_hash{$k}};
1154         my %info_hash;
1155
1156         $info_hash{"email"} = [$k];
1157         $info_hash{"name"} = [$temp{"name"}];
1158
1159         my %sites_hash = %{$temp{"sites"}};
1160         my $l;
1161         my @sites_array;
1162         foreach $l (keys %sites_hash) {
1163             my %sites_xml_hash;
1164             $sites_xml_hash{"url"} = [$l];
1165             if (defined($sites_hash{$l}->{"depth"})) {
1166                 $sites_xml_hash{"depth"} = [$sites_hash{$l}->{"depth"}];
1167             }
1168             if (defined($sites_hash{$l}->{"allow"})) {
1169                 $sites_xml_hash{"allow"} = [$sites_hash{$l}->{"allow"}];
1170             }
1171             push @sites_array, \%sites_xml_hash;
1172         }
1173         $info_hash{"site"} = \@sites_array;
1174         push @professor_array, \%info_hash;
1175     }
1176     $xml_hash{"professor"} = \@professor_array;
1177     return \%xml_hash;
1178 }
1179
1180 sub convertProfessorHashToSiteHash
1181 {
1182     my $hash_ref = shift @_;
1183     my %student_hash = %{$hash_ref};
1184     my %site_hash;
1185
1186     my $v;
1187     foreach $v (values %student_hash) {
1188         die "convertStudentHashToSiteHash: expected <sites>\n" unless
1189             exists($v->{"sites"});
1190         my %url_hash = %{$v->{"sites"}};
1191         my $name = $v->{"name"};
1192
1193         my $url;
1194         foreach $url (keys %url_hash) {
1195             checkUrlFormat($url);
1196             $site_hash{$url} = $url_hash{$url};
1197             $site_hash{$url}->{"name"} = $name;
1198             $SiteTitles{$url} = "";
1199         }
1200     }
1201     return \%site_hash;
1202 }
1203
1204 sub filterProfessorHash
1205 {
1206     my $hash_ref = shift @_;

```

```

1206     my @selectedSites = @_;
1207
1208     my %student_hash = %{$hash_ref};
1209     my %site_hash;
1210
1211     my $k;
1212     foreach $k (keys %student_hash) {
1213         my $v = $student_hash{$k};
1214         die "convertStudentHashToSiteHash: expected <sites>\n" unless
1215             exists($v->{"sites"});
1216         my %url_hash = %{$v->{"sites"}};
1217         my $name = $v->{"name"};
1218
1219         my $url;
1220         my $foundAny = 0;
1221         foreach $url (keys %url_hash) {
1222             #print "Processing $url<br>";
1223             unless (findInArray($url, @selectedSites)) {
1224                 #print "deleting $url<br>";
1225                 delete $url_hash{$url};
1226                 next;
1227             }
1228             $foundAny = 1;
1229         }
1230         if ($foundAny == 0) {
1231             delete $student_hash{$k};
1232         }
1233         else
1234         {
1235             $v->{"sites"} = \%url_hash;
1236         }
1237     }
1238     return \%student_hash;
1239 }
1240
1241 # Looks for the presence of a key in an array.
1242 # Key is the first parameter and @array is the second.
1243 sub findInArray
1244 {
1245     my $key = shift @_;
1246     my @array = @_;
1247     foreach my $k (@array) {
1248         if ($k eq $key) {
1249             return 1;
1250         }
1251     }
1252     return 0;
1253 }
1254
1255 sub setDebug
1256 {
1257     $DEBUG = shift @_;
1258 }
1259
1260 sub debugPrint
1261 {

```

```
1262     my $message = shift @_;
1263     if ($DEBUG == 1) {
1264         print "$message";
1265     }
1266     return 1;
1267 }
1268
1269 sub setBrowser
1270 {
1271     $BROWSER = shift @_;
1272 }
1273
1274 sub browserPrint
1275 {
1276     my $message = shift @_;
1277     if ($BROWSER == 1) {
1278         print "$message";
1279     }
1280     return 1;
1281 }
1282
1283
1284 1;
```

References

- [1] Google web alerts. <http://www.google.com/webalerts>.
- [2] Ibm autonomic computing. <http://www.research.ibm.com/autonomic/>.
- [3] Monitor. <http://hanoi.cs.uiuc.edu/cgi-bin/monitor/research-monitor-admin.pl>.
- [4] Query interface. <http://hanoi.cs.uiuc.edu/cgi-bin/deploy/booksearch.pl>.
- [5] Sigmod. <http://www.db.ucsd.edu/SIGMODPODS03/Organization.html>.
- [6] S. Adali, L. Canadan, Y. Papakonstantinou, and V. Subrahmanian. Query caching and optimization in distributed mediator systems. *In Proceedings of the ACM SIGMOD Conf. on Management of Data*, 1996.
- [7] R. Agrawal, P. Domingos, and M. Richardson. Query caching and optimization in distributed mediator systems. *In Proceedings of the Int. Semantic Web Conference*, 2003.
- [8] N. Ashish and C. Knoblock. Trust management for the semantic web. *Wrapper generation for semi-structured internet resources*, 1997.
- [9] R. Avnur and J. Hellerstein. Continuous query optimization. *In SIGMOD*, 2000.
- [10] V. Boyapati, K. Chevrier, A. Finkel, and N. Glance. Changedetector: A site-level monitoring tool for the www. *In Proceedings of the International Conference on World Wide Web*, 2002.
- [11] K. C. Chang, B. He, C. Li, and Z. Zhang. Structured databases on the web: Observations and implications. Technical Report UIUCDCS-R-2003-2321, Univeristy of Illinois, February 2003.
- [12] J. Chen, D. DeWitt, F. Tian, and Y. Wang. Niagaracq: A scalable continuous query system for internet databases. *In SIGMOD*, 2000.

- [13] A. Doan, P. Domingos, and A. Halevy. Reconciling schemas of disparate data sources: A machine learning approach. *In Proceedings of the ACM SIGMOD Conf. on Management of Data*, 2001.
- [14] A. Doan and McCann R. Building data integration systems: A mass collaboration approach. *In Proceedings of the IJCAI-03 Workshop on Information Integration on the Web*, 2003.
- [15] F. Douhlis, T. Ball, Y. Chen, and E. Koutsofios. The at&t internet difference engine: Tracking and viewing changes on the web. *In World Wide Web*, January 1998.
- [16] O. Etzioni, A. Halevy, A. Doan, Z. Ives, J. Madhavan, L. McDowell, and I. Tatrinov. Crossing the structure chasm. *In Conference of Innovative Database Research*, 2003.
- [17] F. Fabret, F. Llirbat, J. Pereira, and D. Shasha. Efficient matching for content-based publish/subscribe systems. *In Proceedings of Conference on Cooperative Information Systems*, 2000.
- [18] H. Garcia-Molina, Y. Papakonstantinou, D. Quass, A. Rajaraman, Y. Sagiv, J. Ullman, and J. Widom. The tsimmi project: Integration of heterogeneous information sources. *Journal of Intelligent Inf. Systems*, 1997.
- [19] L. M. Haas, D. Kossman, E. L. Wimmers, and J. Yang. Optimizing queries across diverse data sources. *In Proceedings of VLDB*, 1997.
- [20] D. S. Hirshberg. Algorithms for the longest common subsequence problem. *Journal of the ACM*, 1977.
- [21] J. W. Hunt and M. McIlroy. An algorithm for differential file comparison. Technical Report 41, Bell Laboratories, 1975.
- [22] Z. Ives, D. Florescu, M. Friedman, A. Levy, and D. Weld. An adaptive query execution system for data integration. *In Proceedings of SIGMOD*, 1999.
- [23] C. Knoblock, S. Minton, J. Ambite, N. Ashish, P. Modi, I. Muslea, A. Philpot, and S. Tejada. Modeling web sources for information integration. *In the proceedings of the National Conference on Artificial Intelligence (AAAI)*, 1998.
- [24] N. Kushmerick, D. Weld, and R. Doorenbos. Wrapper induction for information extraction. 1997.

- [25] C.T. Kwok and D.S. Weld. Planning to gather information. *In proceedings of the National Conference on Artificial Intelligence (AAAI)*, 1996.
- [26] Y. Levy, A Rajaraman, and J. Ordille. Query answering algorithms for information agents. *In proceedings of the National Conference on Artificial Intelligence (AAAI)*, 1996.
- [27] Y. Levy, A Rajaraman, and J. Ordille. Querying heterogeneous information sources using source descriptions. *In Proceedings of the Int. Conf. on Very Large Data Bases (VLDB)*, 1996.
- [28] R. McCann, A Doan, A Kramnik, and V. Varadarajan. Building data integration systems via mass collaboration. *In Proceedings of the Int. Workshop on Web and Databases (WebDB)*, 2003.
- [29] R. Michalski and G. Tecuci, editors. *Machine Learning: A Multi-strategy approach*. Morgan Kaufmann, 1994.
- [30] R. J. Miller, L.M. Haas, and M. Hernandez. The clio project: Managing heterogeneity. *SIGMOD Record*, 2001.
- [31] S. Pandey, K. Ramamritham, and S. Chakrabarti. Monitoring the dynamic web to respond to continuous queries. *In Proceedings of the International Conference on World Wide Web*, 2003.
- [32] E. Rahm and P. Bernstein. On matching schemas automatically. *VLDB Journal*, 2001.
- [33] R. Ramakrishnan. Mass collaboration and data mining. *KDD*, 2001.
- [34] M. Richardson and P. Domingos. An algorithm for differential file comparison. Technical Report UW-TR-03-02-05, Univ. of Washington, 2003.
- [35] W. Tang, L. Liu, and C. Pu. Webcq: Detecting and delivering information changes on the web. *In Proceedings of the International Conference on Information and Knowledge Management (CIKM)*, 2000.